BZ205 Veri Yapıları ve Algoritmalar

Vize Soruları – Ayrıntılı Çözümler

Hocanın derste anlattığı kurallara göre çözülmüştür

27 Ekim 2026

Çözümlerde uygulanan kurallar (hepsi ders videolarından)

1. **Alfabe:** Türkçe + İngilizce birleşik: A B C Ç D E F G Ğ H I İ J K L M N O Ö P Q R S Ş T U Ü V W X Y Z. Kısa kelime, aynı başlayan uzun kelimeden önce gelir (VAN < VANA). Kelimeler *sayıya çevrilmez.*
2. **Yürütme zamanı** $T(n)$ = temel işlemlerin (atama, karşılaştırma, artırma, aritmetik, return, yazdırma) kaç kez yapıldığı; **zaman karmaşıklığı** = en yüksek dereceli terim: O (en kötü), Ω (en iyi), Θ (ancak ikisi aynı derecedeyse). Değişken tanımları *ihmal edilir* ("ihmal edilmiştir" diye not düşülür). Soruda belirtilmezse *en kötü durum* istenir.
3. **Hızlı sıralama:** soruda verilen pivot kuralı her alt dizide aynen uygulanır; tek dizi üzerinde hocanın "pivotun yerine yaz – pivotun yanındakini boşalt – pivotu kaydır" yöntemi.
4. **Kümeleme sıralama:** *iki aşama* (hip ağacı oluşturma + koparma). Düğüm ≥ çocuklar; zigzag ekleme; koparmada en son eleman köke gelir ve *büyük çocukla* yer değiştirir.
5. **Radiks:** hane sayısı eşitlenir, kelimelerin *sonuna* A eklenir, en sağdaki (önemsiz) haneden başlanır, kovalar geliş sırasını korur.
6. **İkili arama ağacı / AVL silme:** yaprak → sil; tek çocuk → çocuk dedeye bağlanır; iki çocuk → *sağ alt ağacın en küçüğü* yerine geçer. AVL'de her ekleme/silmeden sonra denge kontrolü; fark 2 olunca hemen düzeltilir. Silmede iki problem birden varsa hoca *çift dönüşü* tercih eder.
7. **B+:** iç düğüm max $M-1$, min $\lceil M/2 \rceil - 1$ anahtar; yaprak max $L-1$, min $\lceil L/2 \rceil - 1$ veri. Yaprak taşınca sola $\lfloor L/2 \rfloor$ en küçük, sağa kalanlar; *sağdakinin en küçüğü yukarı kopyalanır.* İç düğüm taşınca $\lceil M/2 \rceil$. anahtar yukarı *taşınır.*

Klasördeki eski çözümlerle farklar

- `Veri Yapıları 2021 Yaz dönemi Vize Sınavı Cozumlu.pdf` dosyasında İAA'dan kök silinirken *solun en büyüğü* (QBE) alınmış. Hoca derste *"Sağ taraftaki minimum eleman silinecek düğümün yerine geçer"* dedi ve WhatsApp'ta da *"biz sağın en küçüğünü alarak işleyeceğiz dedi hoca"* aktarılıyor. Aşağıdaki çözüm hocanın kuralına göredir (QOAW yerine **U** geçer).
- `Veri yapıları Farz edin ki Soruları.pdf` dosyasındaki kodlar **hatalıdır** (`stack.push[j]` ile karşılaştırma, `queue.pop` kullanımı, yığının indisle gezilmesi). WhatsApp'ta da *"Hatalı, veri alırken pop kullanıyoruz"* ve *"o çözümler baya hatalı"* denmiş. Burada doğru kodlar verilmiştir.

İçindekiler

1 2020–2021 Güz Vize Çözümleri 3

1.1 Soru 1a – Kümeleme sıralama (heapsort) . . . 3
1.2 Soru 1b – Hızlı sıralama (pivot: en sondaki kelime) . . . 4
1.3 Soru 2 – AVL ağacı . . . 5
1.4 Soru 3 – B+ ağacı . . . 7
1.5 Soru 4 – Farz edin ki (bağlantılı liste + B+) . . . 9

2 2020–2021 Güz Bütünleme Çözümü (Soru 1 – Kümeleme sıralama) 12

2.1 Soru 1a – Kümeleme sıralama (heapsort) . . . 12
2.2 Soru 1b – Kümeleme sıralamanın akış diyagramı . . . 14
2.3 Soru 1c – Yürütme zamanı ve zaman karmaşıklığı . . . 14

3 2021 Yaz Okulu Vize Çözümleri 16

3.1 Soru 1a – Kabarcık sıralama . . . 16
3.2 Soru 1b – Radiks sıralama . . . 18
3.3 Soru 2 – İkili arama ağacı . . . 19
3.4 Soru 3 – İfade ağacı ve yığın . . . 20
3.5 Soru 4 – Farz edin ki (kuyruk + yığın + İAA) . . . 21

4 2024–2025 Güz Vize Çözümleri 23

4.1 Soru 1 – Genişlik öncelikli arama ve sonrakök (ŞAL'dan, alfabetik sonda öncelikli) . 23
4.2 Soru 2a – Hızlı sıralamanın akış diyagramı . . . 25
4.3 Soru 2b – Yürütme zamanı ve zaman karmaşıklığı . . . 26
4.4 Soru 2c – Hızlı sıralama (pivot: en baştaki kelime) . . . 26
4.5 Soru 3 – B+ ağacı (iç düğüm anahtar sayısı 3, yaprak maks. 3 eleman) . . . 28
4.6 Soru 4 – Farz edin ki (AVL + yığın + bağlantılı liste): sadece bir ders alanlar . . . 30

5 2024–2025 Mazeret Sınavı Çözümleri 31

5.1 Soru 1 – fonk2 ve fonk3 . . . 31
5.2 Soru 2 – TORQ'dan BFS ve sonrakök . . . 32

6 2025–2026 Güz Vize Çözümleri (16 Kasım 2025) 34

6.1 Soru 1 – Hızlı sıralama . . . 34
6.2 Soru 2 – İkili arama ağacında silme . . . 35
6.3 Soru 3 – Farz edin ki (liste + kuyruk + AVL): birden fazla ders alanlar . . . 37
6.4 Soru 4 – AVL ağacı . . . 38

7 Yılı bilinmeyen vize çözümleri 40

7.1 Soru 1 – Hızlı sıralama (örnek kelimeler, pivot en sondaki) . . . 40

7.2 Soru 2 – AVL oluşturma, iki kök silme, öncekök 41
7.3 Soru 3 – Bağlantılı listeye ekleme (InsertNode) 43
7.4 Soru 4 – Farz edin ki (yığın + kuyruk + AVL): üç dersi de alanlar 44

8 Hocanın Örnek Sınav Soruları Çözümleri 44

8.1 Örnek 1 – Şekil 1 AVL: 8 ekle, 2 ekle, 9 sil 44
8.2 Örnek 2 – Şekil 2 AVL: 4 sil; kök sil 46
8.3 Örnek 3 – Graf: DFS ve BFS (A'dan, alfabetik önde öncelikli) 47
8.4 Örnek 4 – AVL (sayılar) 48
8.5 Örnek 5 – Postfix 49
8.6 Örnek 6 – Farz edin ki (liste + kuyruk): her iki dersi alanlar 50

1 2020–2021 Güz Vize Çözümleri

Kelime dizisi (sınavdaki SENA TAŞ YHT örneği): **OWS, TER, NİL, ÖRA, ÜTE, QAE, ŞLK, IRY, XHZ, TEM**. Kendi isminizle oluşturduğunuz dizide adımlar aynı mantıkla yapılır.

1.1 Soru 1a – Kümeleme sıralama (heapsort)

Soru 1 (25 puan) – Sıralama

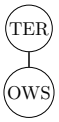
a) Kelime dizisini **kümeleme sıralama (heapsort)** algoritması ile alfabetik olarak (sözlük sırası) sıralayınız. Yapılan her işlem adımını gösteriniz.
b) Kelime dizisini **hızlı sıralama** algoritması ile alfabetik olarak sıralayınız. (*Pivot seçmeniz gerekirse en sondaki kelimeyi pivot seçiniz.*) Yapılan her işlem adımını gösteriniz.

A) Hip (kümeleme) ağacının oluşturulması — zigzag (seviye seviye, soldan sağa) ekleme; her eklemede düğüm ≥ çocukları kontrolü:

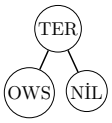
OWS eklendi; hip sağlanıyor



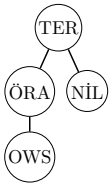
TER eklendi; TER ↔ OWS



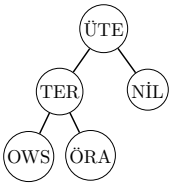
NİL eklendi; hip sağlanıyor



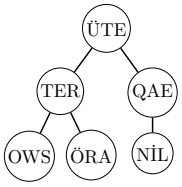
ÖRA eklendi; ÖRA ↔ OWS



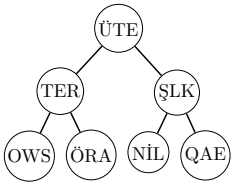
ÜTE eklendi; ÜTE ↔ ÖRA, ÜTE ↔ TER



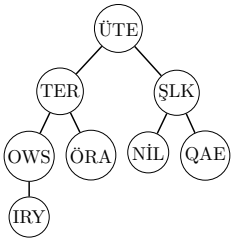
QAE eklendi; QAE ↔ NİL



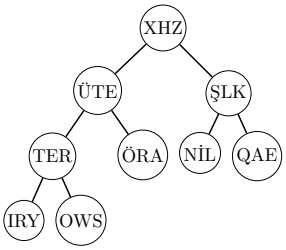
ŞLK eklendi; ŞLK ↔ QAE



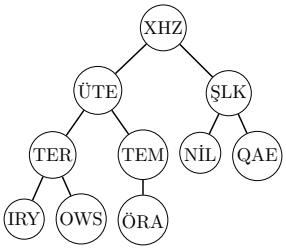
IRY eklendi; hip sağlanıyor



XHZ eklendi; XHZ ↔ OWS, XHZ ↔ TER, XHZ ↔ ÜTE

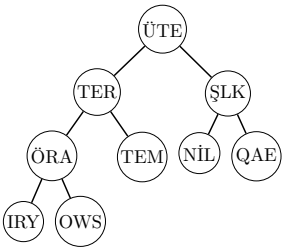


TEM eklendi; TEM ↔ ÖRA

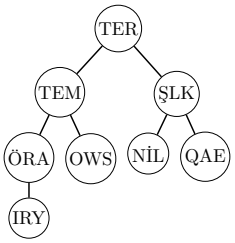


B) Sıralama (koparma) aşaması — kök koparılıp kenara yazılır, zigzag sırasına göre en son eleman köke gelir ve *büyük olan çocukla* yer değiştirerek aşağı iner:

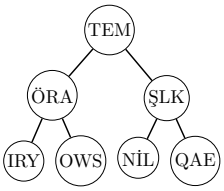
XHZ koparıldı, ÖRA köke geldi;
ÖRA ↔ ÜTE, ÖRA ↔ TER
Kopanlar: XHZ



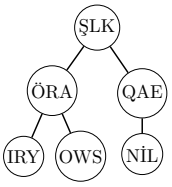
ÜTE koparıldı, OWS köke geldi;
OWS ↔ TER, OWS ↔ TEM
Kopanlar: XHZ, ÜTE



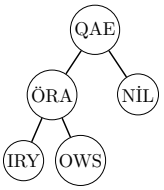
TER koparıldı, IRY köke geldi;
IRY ↔ TEM, IRY ↔ ÖRA
Kopanlar: XHZ, ÜTE, TER



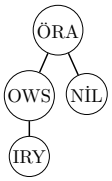
TEM koparıldı, QAE köke geldi;
QAE ↔ ŞLK
Kopanlar: XHZ, ÜTE, TER, TEM



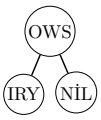
ŞLK koparıldı, NİL köke geldi;
NİL ↔ QAE
Kopanlar: XHZ, ÜTE, TER, TEM, ŞLK



QAE koparıldı, OWS köke geldi;
OWS ↔ ÖRA
Kopanlar: XHZ, ÜTE, TER, TEM, ŞLK, QAE



ÖRA koparıldı, IRY köke geldi;
IRY ↔ OWS
Kopanlar: XHZ, ÜTE, TER, TEM, ŞLK, QAE, ÖRA



OWS koparıldı, NİL köke geldi
Kopanlar: XHZ, ÜTE, TER, TEM, ŞLK, QAE, ÖRA, OWS



NİL koparıldı, IRY köke geldi
Kopanlar: XHZ, ÜTE, TER, TEM, ŞLK, QAE, ÖRA, OWS, NİL



IRY koparıldı
Kopanlar: XHZ, ÜTE, TER, TEM, ŞLK, QAE, ÖRA, OWS, NİL, IRY

Koparılanlar büyükten küçüğe çıktığı için sondan başa yazılır. **Sıralı dizi:** IRY, NİL, OWS, ÖRA, QAE, ŞLK, TEM, TER, ÜTE, XHZ

1.2 Soru 1b – Hızlı sıralama (pivot: en sondaki kelime)

Pivot kuralı: dizinin (alt dizinin) EN SONUNDAKİ eleman pivot seçilir; bu kural *her adımda* aynen uygulanır. Pivottan büyükler sağa, küçükler sola geçer. Yer değiştirme hocanın tek dizi yöntemiyle yapılır: eleman pivotun yerine yazılır, pivotun yanındaki eleman boşalan yere yazılır, pivot bir birim kayar.

Başlangıç: OWS, TER, NİL, ÖRA, ÜTE, QAE, ŞLK, IRY, XHZ, TEM

Alt dizi [0..9], pivot = **TEM**

TER > TEM olduğundan pivotun sağına: OWS, XHZ, NİL, ÖRA, ÜTE, QAE, ŞLK, IRY, **TEM**, TER

XHZ > TEM olduğundan pivotun sağına: OWS, IRY, NİL, ÖRA, ÜTE, QAE, ŞLK, **TEM**, XHZ, TER

ÜTE > TEM olduğundan pivotun sağına: OWS, IRY, NİL, ÖRA, ŞLK, QAE, **TEM**, ÜTE, XHZ, TER

Sonuç: [OWS, IRY, NİL, ÖRA, ŞLK, QAE] **TEM** [ÜTE, XHZ, TER] ⇒ TEM sıralamadaki yerini buldu (indis 6).

Alt dizi [0..5], pivot = **QAE**

ŞLK > QAE olduğundan pivotun sağına: OWS, IRY, NİL, ÖRA, **QAE**, ŞLK

Sonuç: [OWS, IRY, NİL, ÖRA] **QAE** [ŞLK] ⇒ QAE sıralamadaki yerini buldu (indis 4).

Alt dizi [0..3], pivot = **ÖRA**

Hiçbir eleman yer değiştirmedi (tüm elemanlar zaten doğru tarafta).

Sonuç: [OWS, IRY, NİL] **ÖRA** [] ⇒ ÖRA sıralamadaki yerini buldu (indis 3).

Alt dizi [0..2], pivot = **NİL**

OWS > NİL olduğundan pivotun sağına: IRY, **NİL**, OWS

Sonuç: [IRY] **NİL** [OWS] ⇒ NİL sıralamadaki yerini buldu (indis 1).

▷ [0..0] alt dizisinde tek eleman (IRY) kaldı ⇒ yerini buldu.

▷ [2..2] alt dizisinde tek eleman (OWS) kaldı ⇒ yerini buldu.

▷ [5..5] alt dizisinde tek eleman (ŞLK) kaldı ⇒ yerini buldu.

Alt dizi [7..9], pivot = **TER**

ÜTE > TER olduğundan pivotun sağına: XHZ, **TER**, ÜTE

XHZ > TER olduğundan pivotun sağına: **TER**, XHZ, ÜTE

Sonuç: [] **TER** [XHZ, ÜTE] ⇒ TER sıralamadaki yerini buldu (indis 7).

Alt dizi [8..9], pivot = **ÜTE**

XHZ > ÜTE olduğundan pivotun sağına: **ÜTE**, XHZ

Sonuç: [] **ÜTE** [XHZ] ⇒ ÜTE sıralamadaki yerini buldu (indis 8).

▷ [9..9] alt dizisinde tek eleman (XHZ) kaldı ⇒ yerini buldu.

Sıralı dizi: IRY, NİL, OWS, ÖRA, QAE, ŞLK, TEM, TER, ÜTE, XHZ

1.3 Soru 2 – AVL ağacı

Soru 2 (25 puan) – AVL ağacı

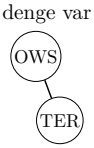
a) Kelime dizisini sırası ile bir **AVL ağacına** yerleştiriniz. Her işlem adımını gösteriniz.
b) a şıkkında oluşan AVL ağacından **kök ard arda iki kez silinirse** ağacın son hali ne olur?
c) a şıkkında oluşan AVL ağacında **sonrakök** gezme yapınız ve kelimelerin hangi sırada gezildiğini listeleyiniz.

Ekleme adımları (her eklemeden sonra eklenen düğümden köke doğru denge kontrolü):

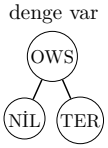
OWS eklendi
denge var



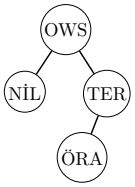
TER eklendi
denge var



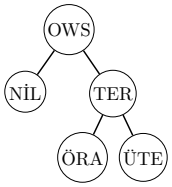
NİL eklendi
denge var



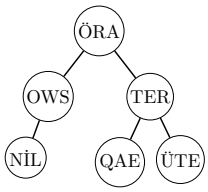
ÖRA eklendi
denge var



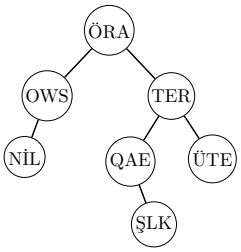
ÜTE eklendi
denge var



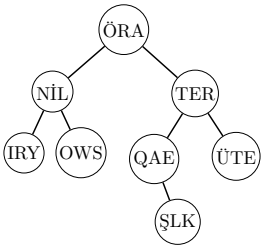
QAE eklendi
OWS düğümünde denge bozuldu:
SAĞ-SOL problemi ⇒ çift dönüş: önce TER'den sağa, sonra OWS'den sola.



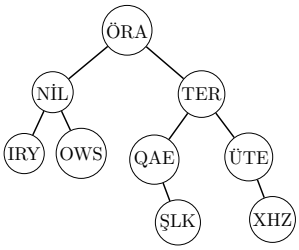
ŞLK eklendi
denge var



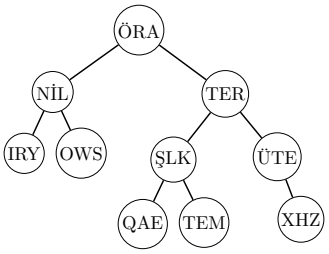
IRY eklendi
OWS düğümünde denge bozuldu:
SOL-SOL problemi ⇒ OWS'den sağa tek dönüş.



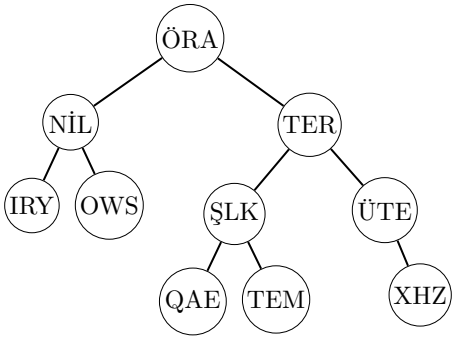
XHZ eklendi
denge var



TEM eklendi
QAE düğümünde denge bozuldu:
SAĞ-SAĞ problemi ⇒ QAE'den sola tek dönüş.



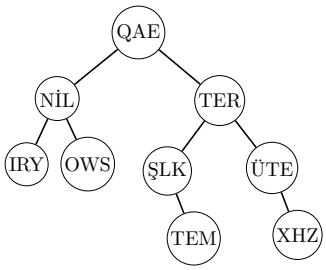
Ağacın son hali (kontrol: içkök gezme sıralı verir: IRY, NİL, OWS, ÖRA, QAE, ŞLK, TEM, TER, ÜTE, XHZ):



Silme adımları (iki çocuklu düğüm silinirken *sağ alt ağacın en küçüğü* yerine geçer; ardından denge kontrolü):

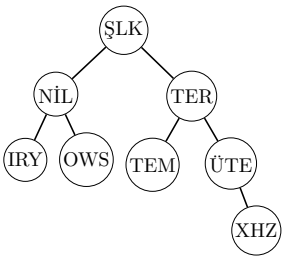
1. kök silme: **ÖRA**

ÖRA iki çocuklu: sağ alt ağacın en küçüğü QAE köke/yerine geçer. Ardından QAE sağ alt ağaçtaki eski yerinden silinir: QAE yaprak, doğrudan silinir. Denge bozulmadı.



2. kök silme: **QAE**

QAE iki çocuklu: sağ alt ağacın en küçüğü ŞLK köke/yerine geçer. Ardından ŞLK sağ alt ağaçtaki eski yerinden silinir: ŞLK tek çocuklu, çocuğu TEM yukarı bağlanır. Denge bozulmadı.



Sonrakök (postorder: sol–sağ–kök) gezme (ekleme sonrası oluşan ağaç üzerinde): **IRY, OWS, NİL, QAE, TEM, ŞLK, XHZ, ÜTE, TER, ÖRA**

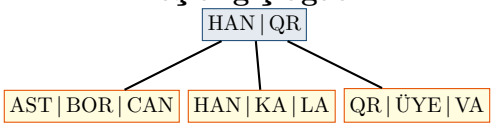
1.4 Soru 3 – B+ ağacı

Soru 3 (25 puan) – B+ ağacı

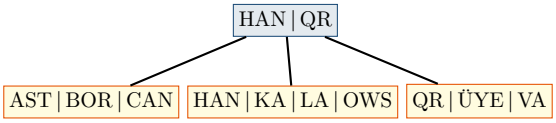
Ağacın iç düğüm çocuk sayısı 4, yapraklarda tutulan maksimum veri sayısı 4'tür. Başlangıç ağacı: kök [HAN | QR]; yapraklar [AST, BOR, CAN], [HAN, KA, LA], [QR, ÜYE, VA].
a) Kelime dizisini sırası ile B+ ağacına yerleştiriniz. b) Sırasıyla QR ve ÜYE silinirse değişimleri gösteriniz. c) Ardından HAN ve VA silinirse ağacın son halini çiziniz.

Soruda "iç düğüm çocuk sayısı 4" $\Rightarrow M = 4$; "yapraklarda tutulan maksimum veri 4" $\Rightarrow L - 1 = 4$, yani $L = 5$. Parametreler: iç düğüm çocuk sayısı $M = 4 \Rightarrow$ iç düğümde en fazla $M - 1 = 3$, en az $\lceil M/2 \rceil - 1 = 1$ anahtar; $L = 5 \Rightarrow$ yaprakta en fazla $L - 1 = 4$, en az $\lceil L/2 \rceil - 1 = 2$ veri.

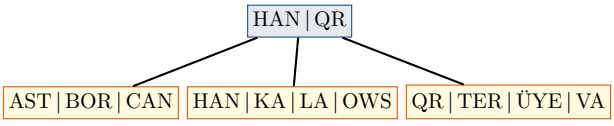
Başlangıç ağacı



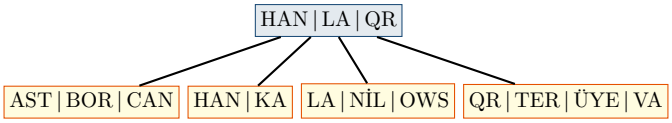
OWS ekle: OWS yaprağa eklendi (yaprakta boş yer vardı).



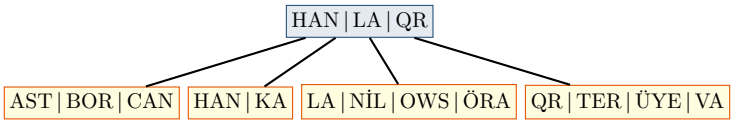
TER ekle: TER yaprağa eklendi (yaprakta boş yer vardı).



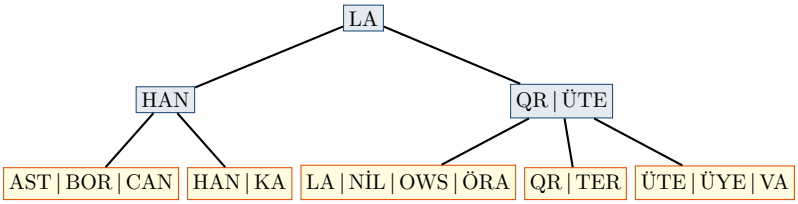
NİL ekle: NİL eklenince yaprakta 5 veri oldu (maks. 4) $\Rightarrow$ yaprak bölünür: sola $\lfloor 5/2 \rfloor$=2 en küçük (HAN, KA), sağa 3 (LA, NİL, OWS); sağdakinin en küçüğü LA yukarı anahtar olarak KOPYALANIR.



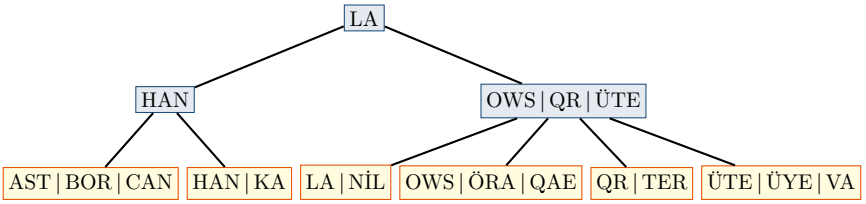
ÖRA ekle: ÖRA yaprağa eklendi (yaprakta boş yer vardı).



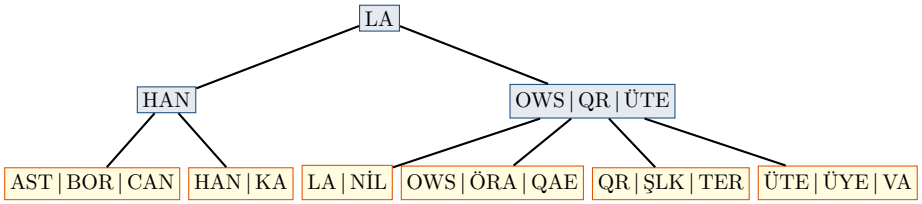
ÜTE ekle: ÜTE eklenince yaprakta 5 veri oldu (maks. 4) ⇒ yaprak bölünür: sola $\lfloor 5/2 \rfloor$=2 en küçük (QR, TER), sağa 3 (ÜTE, ÜYE, VA); sağdakinin en küçüğü ÜTE yukarı anahtar olarak KOPYALANIR. İç düğümde 4 anahtar oldu (maks. 3) ⇒ iç düğüm bölünür: sola $\lceil 4/2 \rceil - 1$=1 anahtar (HAN), LA yukarı TAŞINIR, sağa QR, ÜTE. Kök bölündüğü için yeni kök oluştu: [LA].



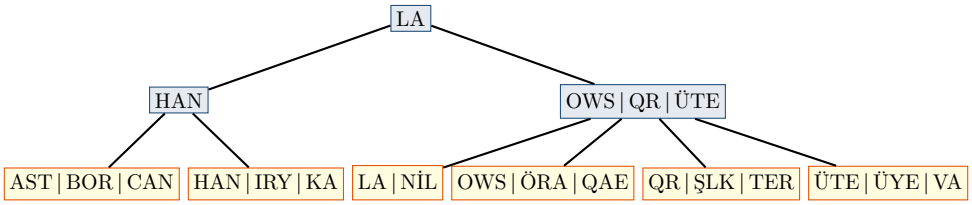
QAE ekle: QAE eklenince yaprakta 5 veri oldu (maks. 4) ⇒ yaprak bölünür: sola $\lfloor 5/2 \rfloor$=2 en küçük (LA, NİL), sağa 3 (OWS, ÖRA, QAE); sağdakinin en küçüğü OWS yukarı anahtar olarak KOPYALANIR.



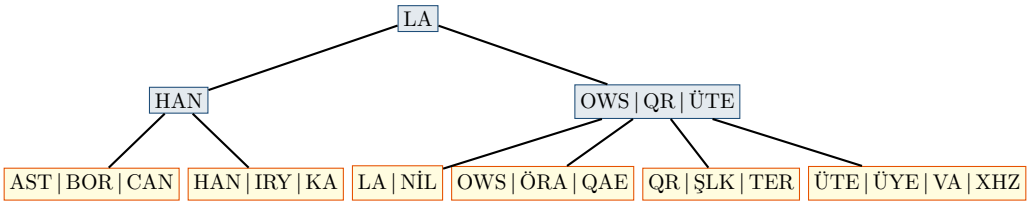
ŞLK ekle: ŞLK yaprağa eklendi (yaprakta boş yer vardı).



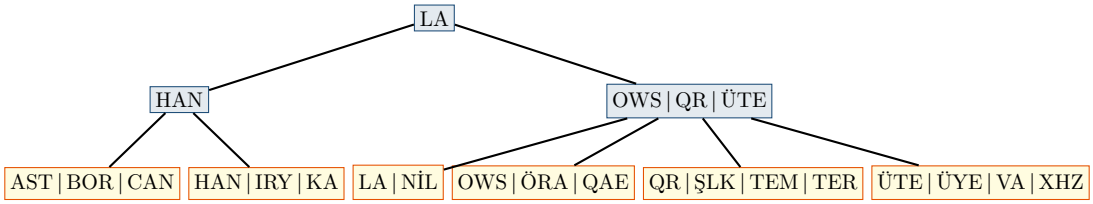
IRY ekle: IRY yaprağa eklendi (yaprakta boş yer vardı).



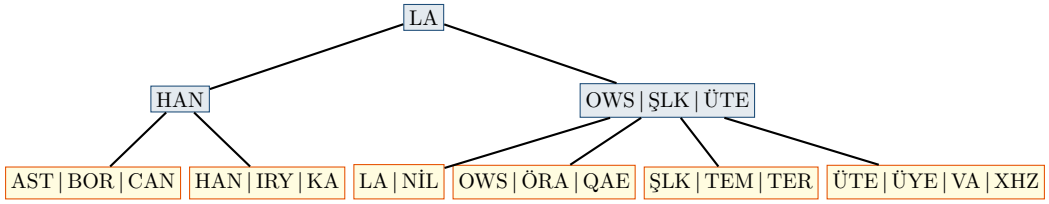
XHZ ekle: XHZ yaprağa eklendi (yaprakta boş yer vardı).



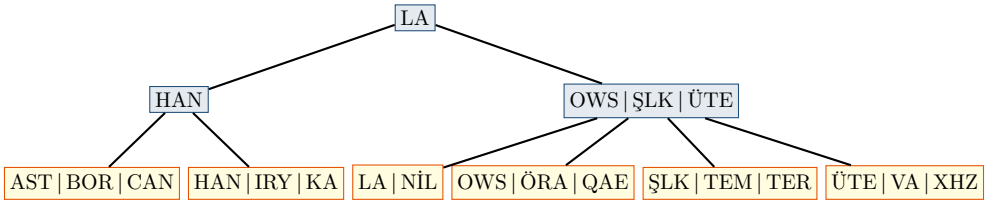
TEM ekle: TEM yaprağa eklendi (yaprakta boş yer vardı).



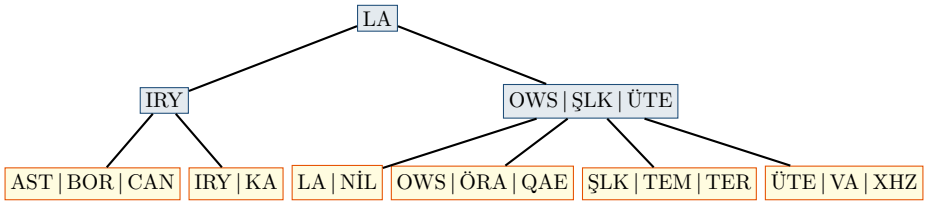
QR sil: QR yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



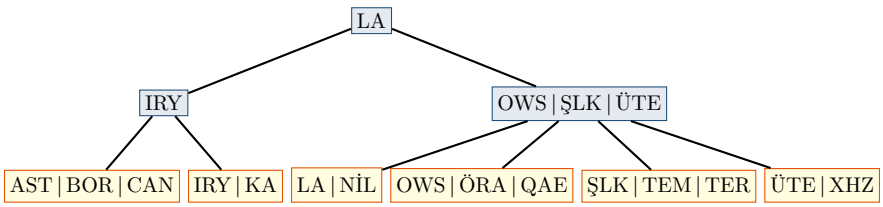
ÜYE sil: ÜYE yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



HAN sil: HAN yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



VA sil: VA yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



1.5 Soru 4 – Farz edin ki (bağlantılı liste + B+)

Soru 4 (25 puan) – Farz edin ki

Farz edin ki bir **bağlantılı listede** İstatistik dersini alan öğrenciler, bir **B+ ağacında** Matematik dersini alan öğrenciler tutulmaktadır. Her iki derste de numara, ad ve soyad tutulur. B+ ağacında indeksleme öğrenci numarasına göredir.
a) **Sadece Matematik** dersini alan öğrencileri listeleyen fonksiyonun kaba kodunu yazınız (temel bağlantılı liste ve ağaç işlemlerini kullanınız). b) Mantığını açıklayınız. c) Akış diyagramını çiziniz. d) Yürütme zamanını ve zaman karmaşıklığını hesaplayınız.
Not1: Her iki yapıda da öğrenciler sıralı değildir. Not2: Fonksiyonun sonunda sınıf listeleri değişmeyecektir.

İstenen: B+ ağacında (Matematik) olup bağlantılı listede (İstatistik) *olmayan* öğrenciler.

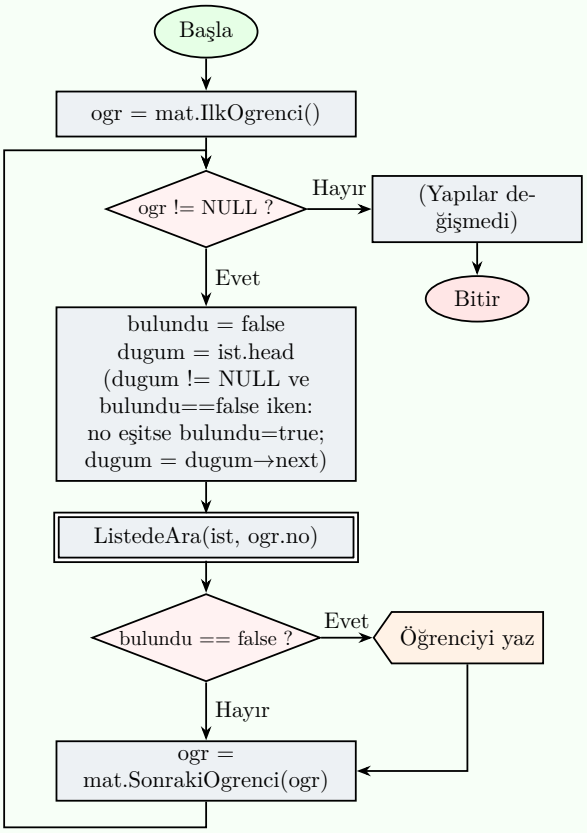
a) Kaba kod

```
// mat : B+ agaci (ogrenci numarasina gore indeksli), ist : bagli liste
// Ogrenci: no, ad, soyad.  Degisken tanimlari ihmal edilmistir.
SadeceMatematik(BPlus mat, List ist)
 1  ogr = mat.IlkOgrenci()              // en soldaki yapragin ilk verisi
 2  while (ogr != NULL)                 // B+ yapraklari soldan saga gezilir
 3     bulundu = false
 4     dugum = ist.head
 5     while (dugum != NULL && bulundu == false)
 6        if (dugum->ogr.no == ogr.no)
 7           bulundu = true
 8        dugum = dugum->next
 9     if (bulundu == false)
10        Yaz(ogr.no, ogr.ad, ogr.soyad)
11     ogr = mat.SonrakiOgrenci(ogr)   // yapraktaki sonraki veri / sonraki yaprak
```

b) Mantık

B+ ağacının bütün verileri yapraklardadır ve yapraklar soldan sağa birbirine bağlıdır; bu yüzden Matematik öğrencileri yapraklar üzerinden *tek tek* gezilir (satır 1, 2, 11). Her Matematik öğrencisi için bağlantılı liste `head`'den başlanarak `next` ile sonuna kadar gezilir (satır 4–8); numarası eşleşirse öğrenci İstatistik de alıyordur ve iç döngü durur. Listede bulunamayan öğrenci *sadece Matematik* alıyordur ve yazdırılır (satır 9–10). Hiçbir yapıdan eleman silinmediği / eklenmediği için sınıf listeleri değişmez (Not2). Liste sırasız olduğu için listede arama doğrusal yapılmak zorundadır (Not1).

c) Akış diyagramı



Listede arama kutusu (çift kenarlı = fonksiyon) ayrıca açılırsa satır 4–8'deki iç döngüdür; hocanın kuralına göre kendi yazdığımız fonksiyonun detayı ayrı bir akış diyagramı olarak verilir.

d) Yürütme zamanı ve zaman karmaşıklığı

m: Matematik (B+) öğrenci sayısı, n: İstatistik (liste) öğrenci sayısı. **En kötü durum:** iki dersin ortak öğrencisi yok (iç döngü her seferinde listenin sonuna kadar gider, satır 7 hiç çalışmaz, satır 10 her seferinde çalışır).

Satır	Açıklama	Tekrar
1	ilk öğrenciyi al	1
2	dış döngü kontrolü	$m+1$
3, 4	atama	$m+m$
5	iç döngü kontrolü	$m(n+1)$
6	karşılaştırma	mn
7	atama	0
8	ilerleme	mn
9	kontrol	m
10	yazdırma	m
11	sonraki öğrenci	m

$T(m,n) = 1 + (m+1) + 2m + (mn+m) + mn + mn + m + m + m = \mathbf{3mn + 7m + 2} \quad \Rightarrow$ $O(m \cdot n)$; $m = n$ ise $O(n^2)$.
En iyi durum: liste boş ($n = 0$) $\Rightarrow T = 7m+2$, $\Omega(m)$. Dereceler farklı olduğu için Θ verilemez.

2 2020–2021 Güz Bütünleme Çözümü (Soru 1 – Kümeleme sıralama)

Soru 1 (25 puan)

a) Yukarıda 2. adımda oluşturmuş olduğunuz kelime dizisini kümeleme sıralama (heapsort) algoritması ile alfabetik olarak (sözlük sırası) sıralayınız? Yapılan her işlem adımını gösteriniz?
b) Kümeleme sıralamanın akış diyagramını çiziniz?
c) Kümeleme sıralamanın yürütme zamanını ve zaman karmaşıklığını hesap ediniz? Hesaplama adımlarını gösteriniz?

Kelime kalıbı: `_Q R_ _S _N T_ _O _U Ü_ X_ P_`. Sınavdaki örnek isim dizisiyle (SENA TAŞ + rastgele Y H T) kelime dizisi: **SQ, RE, NS, AN, TT, AO, ŞU, ÜY, XH, PT**. Kendi isminizle oluşturduğunuz dizide adımlar aynı mantıkla yapılır.

Bu dizide karıştırılabilecek karşılaştırmalar

Alfabe: A B C Ç D E F G Ğ H I İ J K L M N O Ö P Q R S Ş T U Ü V W X Y Z. **AN** < **AO** (ilk harf aynı, ikinci harfe bakılır: N, O'dan önce); **SQ** < **ŞU** (S, Ş'den önce); **ŞU** < **TT** (Ş, T'den önce); **PT** < **RE** (P, R'den önce; Q, P ile R arasındadır).

2.1 Soru 1a – Kümeleme sıralama (heapsort)

A) Hip (kümeleme) ağacının oluşturulması — zigzag (seviye seviye, soldan sağa) ekleme; her eklemede düğüm ≥ çocukları kontrolü ("büyük" = alfabede sonra gelen):

SQ eklendi; hip sağlanıyor



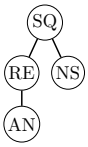
RE eklendi; hip sağlanıyor



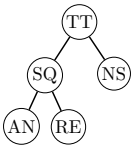
NS eklendi; hip sağlanıyor



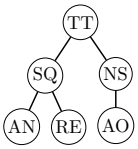
AN eklendi; hip sağlanıyor



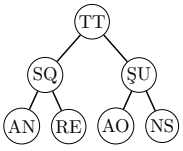
TT eklendi; TT ↔ RE, TT ↔ SQ



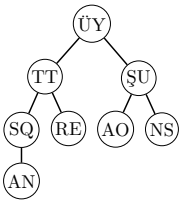
AO eklendi; hip sağlanıyor



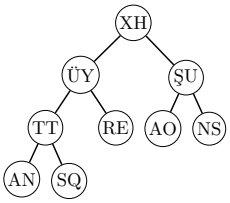
ŞU eklendi; ŞU ↔ NS



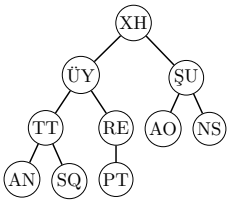
ÜY eklendi; ÜY ↔ AN, ÜY ↔ SQ, ÜY ↔ TT



XH eklendi; XH ↔ SQ, XH ↔ TT, XH ↔ ÜY



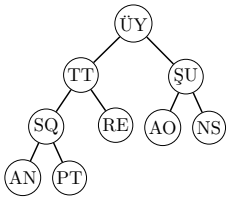
PT eklendi; hip sağlanıyor



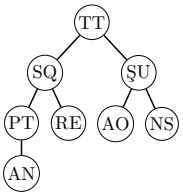
Eklenen	Yer değiştirmeler	H (zigzag sırası)
SQ	–	`SQ`
RE	–	`SQ RE`
NS	–	`SQ RE NS`
AN	–	`SQ RE NS AN`
TT	TT ↔ RE, TT ↔ SQ	`TT SQ NS AN RE`
AO	–	`TT SQ NS AN RE AO`
ŞU	ŞU ↔ NS	`TT SQ ŞU AN RE AO NS`
ÜY	ÜY ↔ AN, ÜY ↔ SQ, ÜY ↔ TT	`ÜY TT ŞU SQ RE AO NS AN`
XH	XH ↔ SQ, XH ↔ TT, XH ↔ ÜY	`XH ÜY ŞU TT RE AO NS AN SQ`
PT	–	`XH ÜY ŞU TT RE AO NS AN SQ PT`

B) Sıralama (koparma) aşaması — kök koparılıp kenara yazılır, zigzag sırasına göre en son eleman köke gelir ve *büyük olan çocukla* yer değiştirerek aşağı iner:

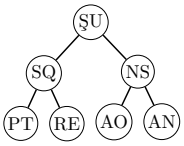
XH koparıldı, PT köke geldi; PT ↔ ÜY, PT ↔ TT, PT ↔ SQ
Kopanlar: XH



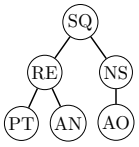
ÜY koparıldı, PT köke geldi; PT ↔ TT, PT ↔ SQ (PT ≥ AN, dur)
Kopanlar: XH, ÜY



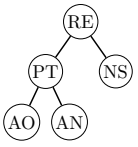
TT koparıldı, AN köke geldi; AN ↔ ŞU, AN ↔ NS
Kopanlar: XH, ÜY, TT



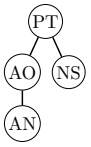
ŞU koparıldı, AN köke geldi; AN ↔ SQ, AN ↔ RE
Kopanlar: XH, ÜY, TT, ŞU



SQ koparıldı, AO köke geldi; AO ↔ RE, AO ↔ PT
Kopanlar: XH, ÜY, TT, ŞU, SQ



RE koparıldı, AN köke geldi; AN ↔ PT, AN ↔ AO
Kopanlar: XH, ÜY, TT, ŞU, SQ, RE



PT koparıldı, AN köke geldi; AN ↔ NS
Kopanlar: XH, ÜY, TT, ŞU, SQ, RE, PT



NS koparıldı, AN köke geldi; AN ↔ AO
Kopanlar: XH, ÜY, TT, ŞU, SQ, RE, PT, NS



AO koparıldı, AN köke geldi
Kopanlar: XH, ÜY, TT, ŞU, SQ, RE, PT, NS, AO



AN koparıldı
Kopanlar: XH, ÜY, TT, ŞU, SQ, RE, PT, NS, AO, AN

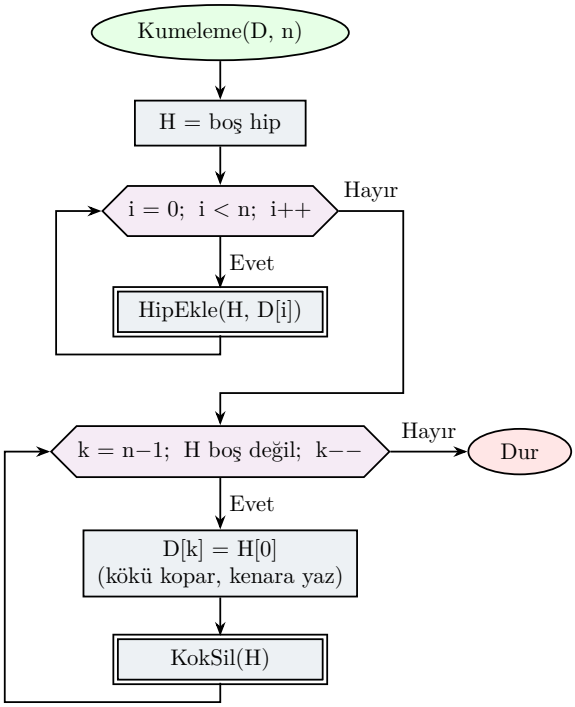


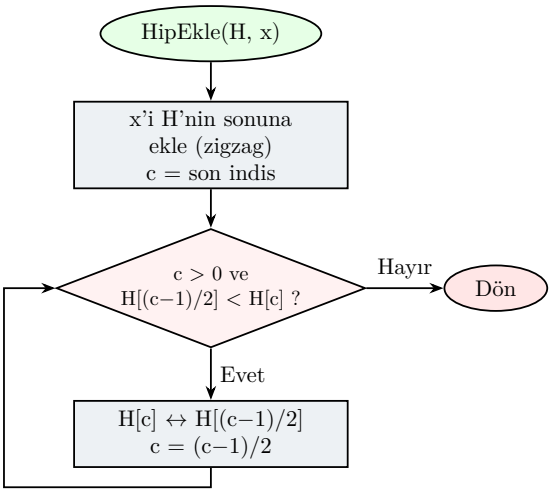
Koparılan	Köke gelen	Yer değiştirmeler	Kalan H
XH	PT	PT ↔ ÜY, PT ↔ TT, PT ↔ SQ	`ÜY TT ŞU SQ RE AO NS AN PT`
ÜY	PT	PT ↔ TT, PT ↔ SQ (PT ≥ AN, dur)	`TT SQ ŞU PT RE AO NS AN`
TT	AN	AN ↔ ŞU, AN ↔ NS	`ŞU SQ NS PT RE AO AN`
ŞU	AN	AN ↔ SQ, AN ↔ RE	`SQ RE NS PT AN AO`
SQ	AO	AO ↔ RE, AO ↔ PT	`RE PT NS AO AN`
RE	AN	AN ↔ PT, AN ↔ AO	`PT AO NS AN`
PT	AN	AN ↔ NS	`NS AO AN`
NS	AN	AN ↔ AO	`AO AN`
AO	AN	–	`AN`
AN	–	–	`(boş)`

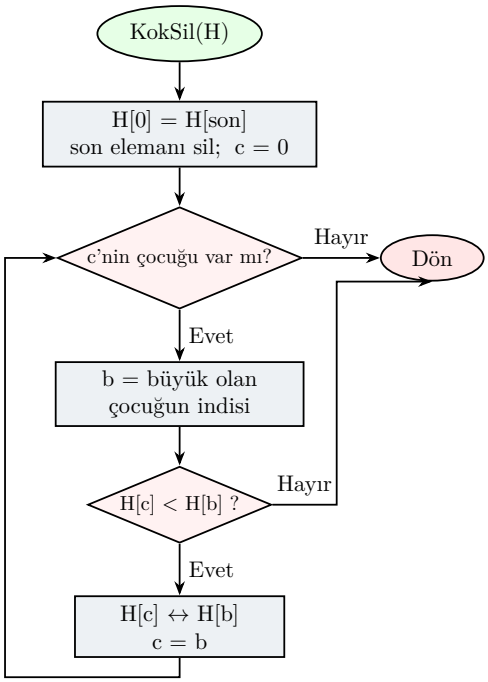
Koparılanlar büyükten küçüğe çıktığı için sondan başa yazılır. **Sıralı dizi:** AN, AO, NS, PT, RE, SQ, ŞU, TT, ÜY, XH

2.2 Soru 1b – Kümeleme sıralamanın akış diyagramı

Kendi yazdığımız `HipEkle` ve `KokSil` fonksiyonlarını çağırdığımız için onların akış diyagramları da verilir: *"o fonksiyonun detaylarını veya akış diyagramını vermemiz gerekecek"* [video 2 Akış Diyagramı 58:33]. Ana diyagramda *iki döngü* (oluşturma ve koparma) görünmelidir; ağacı oluşturup bırakmak eksik cevaptır. Kelimelerde "</>" karşılaştırması "alfabede önce/sonra gelir" anlamındadır.







2.3 Soru 1c – Yürütme zamanı ve zaman karmaşıklığı

Satır numaralı kod (hocanın slayt stilinde; değişken tanımlamaları ihmal edilmiştir; kelimelerde > = "alfabede sonra gelir"):

```
Kumeleme ( D[], N)
 1   for (i=0; i<N; i++)              // A) agac olusturma
 2       HipEkle(H, i, D[i]);
 3   for (k=N-1; k>=0; k--) {         // B) siralama
 4       D[k] = H[0];                 // koku kopar, kenara yaz
 5       KokSil(H, k);
     }

HipEkle ( H[], son, x)
 6   H[son] = x;
 7   c = son;
 8   ebeveyn = (c-1)/2;
 9   while (c>0 && H[c]>H[ebeveyn]) {
10       gecici = H[c];
11       H[c] = H[ebeveyn];
12       H[ebeveyn] = gecici;
13       c = ebeveyn;
14       ebeveyn = (c-1)/2;
     }

KokSil ( H[], son)
15   H[0] = H[son];
16   son = son-1;
17   c = 0;
18   while (2*c+1 <= son) {
19       buyuk = 2*c+1;
20       if (2*c+2 <= son && H[2*c+2] > H[buyuk])
21           buyuk = 2*c+2;
22       if (H[c] >= H[buyuk])
23           break;
24       gecici = H[c];
25       H[c] = H[buyuk];
26       H[buyuk] = gecici;
27       c = buyuk;
     }
```

Çözüm

Tanımlar: n = kelime sayısı; her ifade 1 işlem, iki kelimenin karşılaştırılması 1 işlem; `for (i=0; i<N; i++)` $= 1 + (n+1) + n = 2n + 2$; `while` koşulu gövdeye giriş sayısından 1 fazla kontrol edilir. u = bir eklemedeki yukarı, d = bir koparmadaki aşağı yer değiştirme sayısı.
Ağacın yüksekliği: zigzag ile her seviye bir öncekinin iki katı eleman alır (1, 2, 4, …), $2^h = n \Rightarrow h = \log_2 n$. Bir eleman en fazla yükseklik kadar çıkar/iner: $u, d \leq \log_2 n$.
En kötü durum: her eklenen kelime köke kadar çıkar (kelimeler alfabetik sırada gelir), her koparmada köke gelen yaprağa kadar iner ve sağ çocuk hep büyüktür ($u = d = \log_2 n$, satır 23 hiç çalışmaz).

Satır	İşlem	Tekrar (en kötü)
1	dış for (oluşturma)	$2n+2$
2	HipEkle çağrısı	n
3	dış for (koparma)	$2n+2$
4	kökü kenara yaz	n
5	KokSil çağrısı	n
6, 7, 8	3 atama	$3n$
9	while kontrolü	$n(\log_2 n+1)$
10–14	yer değiştirme + ilerleme	$5n\log_2 n$
15, 16, 17	3 atama	$3n$
18	while kontrolü	$n(\log_2 n+1)$
19–22	büyük çocuğu seç + kontrol	$4n\log_2 n$
23	break	0
24–27	yer değiştirme + ilerleme	$4n\log_2 n$

Ana fonksiyon: $7n+4$; n tane HipEkle: $n(4+6\log_2 n)$; n tane KokSil: $n(4+9\log_2 n)$.

$$T(n) = (7n+4) + (6n\log_2 n + 4n) + (9n\log_2 n + 4n) = \mathbf{15\,n\log_2 n + 15n + 4}$$

En iyi durum: kelimeler ters alfabetik sırada gelirse oluşturmada hiç yer değiştirme olmaz ($u=0$, oluşturma $7n+2$, doğrusal). Koparma ise girdiden bağımsızdır: köke her seferinde en alt seviyeden küçük bir eleman gelir ve yine yaklaşık $\log_2 n$ seviye iner $\Rightarrow$ toplam yine $n\log_2 n$ mertebesinde.
Zaman karmaşıklığına geçiş: en büyük dereceli terim $n\log_2 n$; $15n$ ve 4 ihmal edilir, katsayı 15 atılır, logaritmanın tabanı sabit çarpan olduğu için önemsizdir. En kötü $\mathbf{O(n\log n)}$, en iyi $(\mathbf{n\log n})$; ikisi aynı derecede olduğu için $(\mathbf{n\log n})$. Ek alan: hip ayrı `H` dizisinde tutulur, $\mathbf{O(n)}$. Hocanın gerekçesi: *"Ağacın yüksekliği…log n…her bir n tane eleman için işlemi yapacağız. Dolayısıyla n log n"* [video 6 Sıralama 3].
Bu örnek için kontrol ($n=10$): a şıkkında 9 yukarı (TT 2, ŞU 1, ÜY 3, XH 3) ve 15 aşağı yer değiştirme var; aynı kurallarla sayınca $T=337$ işlem, formülün üst sınırı $15\cdot10\cdot3{,}32+154\approx652$. Fark, ağaç başta küçük olduğu için ilk eklemelerin $\log_2 n$ kadar yükselememesinden gelir; sonuç yine $n\log n$.

3 2021 Yaz Okulu Vize Çözümleri

Kelime dizisi (çözümlü PDF'teki): **QOAW, HTR, LMİ, EÖR, PTE, QBE, U, IĞR, XRZ, XAEM**.

3.1 Soru 1a – Kabarcık sıralama

Soru 1 – Sıralama

a) Kelime dizisini **kabarcık sıralama (bubblesort)** ile alfabetik olarak sıralayınız; her adımı gösteriniz.
b) Kelime dizisini **radiks sıralama (radixsort)** ile alfabetik olarak sıralayınız; her adımı gösteriniz.

Dış döngü (hareket) = dalga; her dalgada iç döngü $k=0\ldots n-2$ – hareket komşuları karşılaştırır, büyük olan sağa geçer.

Hareket	k	İşlem	Dizi (işlem sonrası)
0	0	QOAW ↔ HTR	**HTR**, **QOAW**, LMİ, EÖR, PTE, QBE, U, IĞR, XRZ, XAEM
0	1	QOAW ↔ LMİ	HTR, **LMİ**, **QOAW**, EÖR, PTE, QBE, U, IĞR, XRZ, XAEM
0	2	QOAW ↔ EÖR	HTR, LMİ, **EÖR**, **QOAW**, PTE, QBE, U, IĞR, XRZ, XAEM
0	3	QOAW ↔ PTE	HTR, LMİ, EÖR, **PTE**, **QOAW**, QBE, U, IĞR, XRZ, XAEM
0	4	QOAW ↔ QBE	HTR, LMİ, EÖR, PTE, **QBE**, **QOAW**, U, IĞR, XRZ, XAEM
0	5	değişim yok	HTR, LMİ, EÖR, PTE, QBE, QOAW, U, IĞR, XRZ, XAEM
0	6	U ↔ IĞR	HTR, LMİ, EÖR, PTE, QBE, QOAW, **IĞR**, **U**, XRZ, XAEM
0	7	değişim yok	HTR, LMİ, EÖR, PTE, QBE, QOAW, IĞR, U, XRZ, XAEM
0	8	XRZ ↔ XAEM	HTR, LMİ, EÖR, PTE, QBE, QOAW, IĞR, U, **XAEM**, **XRZ**
1	0	değişim yok	HTR, LMİ, EÖR, PTE, QBE, QOAW, IĞR, U, XAEM, XRZ
1	1	LMİ ↔ EÖR	HTR, **EÖR**, **LMİ**, PTE, QBE, QOAW, IĞR, U, XAEM, XRZ
1	2	değişim yok	HTR, EÖR, LMİ, PTE, QBE, QOAW, IĞR, U, XAEM, XRZ
1	3	değişim yok	HTR, EÖR, LMİ, PTE, QBE, QOAW, IĞR, U, XAEM, XRZ
1	4	değişim yok	HTR, EÖR, LMİ, PTE, QBE, QOAW, IĞR, U, XAEM, XRZ
1	5	QOAW ↔ IĞR	HTR, EÖR, LMİ, PTE, QBE, **IĞR**, **QOAW**, U, XAEM, XRZ
1	6	değişim yok	HTR, EÖR, LMİ, PTE, QBE, IĞR, QOAW, U, XAEM, XRZ
1	7	değişim yok	HTR, EÖR, LMİ, PTE, QBE, IĞR, QOAW, U, XAEM, XRZ
2	0	HTR ↔ EÖR	**EÖR**, **HTR**, LMİ, PTE, QBE, IĞR, QOAW, U, XAEM, XRZ
2	1	değişim yok	EÖR, HTR, LMİ, PTE, QBE, IĞR, QOAW, U, XAEM, XRZ
2	2	değişim yok	EÖR, HTR, LMİ, PTE, QBE, IĞR, QOAW, U, XAEM, XRZ
2	3	değişim yok	EÖR, HTR, LMİ, PTE, QBE, IĞR, QOAW, U, XAEM, XRZ
2	4	QBE ↔ IĞR	EÖR, HTR, LMİ, PTE, **IĞR**, **QBE**, QOAW, U, XAEM, XRZ
2	5	değişim yok	EÖR, HTR, LMİ, PTE, IĞR, QBE, QOAW, U, XAEM, XRZ
2	6	değişim yok	EÖR, HTR, LMİ, PTE, IĞR, QBE, QOAW, U, XAEM, XRZ
3	0	değişim yok	EÖR, HTR, LMİ, PTE, IĞR, QBE, QOAW, U, XAEM, XRZ
3	1	değişim yok	EÖR, HTR, LMİ, PTE, IĞR, QBE, QOAW, U, XAEM, XRZ
3	2	değişim yok	EÖR, HTR, LMİ, PTE, IĞR, QBE, QOAW, U, XAEM, XRZ
3	3	PTE ↔ IĞR	EÖR, HTR, LMİ, **IĞR**, **PTE**, QBE, QOAW, U, XAEM, XRZ
3	4	değişim yok	EÖR, HTR, LMİ, IĞR, PTE, QBE, QOAW, U, XAEM, XRZ
3	5	değişim yok	EÖR, HTR, LMİ, IĞR, PTE, QBE, QOAW, U, XAEM, XRZ
4	0	değişim yok	EÖR, HTR, LMİ, IĞR, PTE, QBE, QOAW, U, XAEM, XRZ

Hareket	k	İşlem	Dizi (işlem sonrası)
4	1	değişim yok	EÖR, HTR, LMİ, IĞR, PTE, QBE, QOAW, U, XAEM, XRZ
4	2	LMİ ↔ IĞR	EÖR, HTR, **IĞR**, **LMİ**, PTE, QBE, QOAW, U, XAEM, XRZ
4	3	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
4	4	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
5	0	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
5	1	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
5	2	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
5	3	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
6	0	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
6	1	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
6	2	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
7	0	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
7	1	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ
8	0	değişim yok	EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ

Sonuç: EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ

3.2 Soru 1b – Radiks sıralama

Hane eşitleme (en uzun kelime 4 harf; sözlük sırası bozulmasın diye kelimelerin **sonuna** 'A' eklenir): QOAW, HTRA, LMİA, EÖRA, PTEA, QBEA, UAAA, IĞRA, XRZA, XAEM

1. geçiş (sağdan 1. harfe göre kovalar): **A**: HTRA, LMİA, EÖRA, PTEA, QBEA, UAAA, IĞRA, XRZA; **M**: XAEM; **W**: QOAW

⇒ HTRA, LMİA, EÖRA, PTEA, QBEA, UAAA, IĞRA, XRZA, XAEM, QOAW

2. geçiş (sağdan 2. harfe göre kovalar): **A**: UAAA, QOAW; **E**: PTEA, QBEA, XAEM; **İ**: LMİA; **R**: HTRA, EÖRA, IĞRA; **Z**: XRZA

⇒ UAAA, QOAW, PTEA, QBEA, XAEM, LMİA, HTRA, EÖRA, IĞRA, XRZA

3. geçiş (sağdan 3. harfe göre kovalar): **A**: UAAA, XAEM; **B**: QBEA; **Ğ**: IĞRA; **M**: LMİA; **O**: QOAW; **Ö**: EÖRA; **R**: XRZA; **T**: PTEA, HTRA

⇒ UAAA, XAEM, QBEA, IĞRA, LMİA, QOAW, EÖRA, XRZA, PTEA, HTRA

4. geçiş (sağdan 4. harfe göre kovalar): **E**: EÖRA; **H**: HTRA; **I**: IĞRA; **L**: LMİA; **P**: PTEA; **Q**: QBEA, QOAW; **U**: UAAA; **X**: XAEM, XRZA

⇒ EÖRA, HTRA, IĞRA, LMİA, PTEA, QBEA, QOAW, UAAA, XAEM, XRZA

Eklenen A'lar atılır. **Sonuç:** EÖR, HTR, IĞR, LMİ, PTE, QBE, QOAW, U, XAEM, XRZ

3.3 Soru 2 – İkili arama ağacı

Soru 2 – İkili arama ağacı

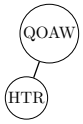
a) Kelime dizisini sıra ile bir **ikili arama ağacına** yerleştiriniz; her adımı gösteriniz.
b) Ağaçtan **kök ard arda iki kez silinirse** ağacın son hali ne olur?
c) b şıkkındaki ağaca sırasıyla **EQN, HM, GZT** eklenirse ağacın son hali ne olur?

Ekleme adımları (küçükler sola, büyükler sağa; her kademede kontrol):

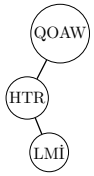
QOAW eklendi



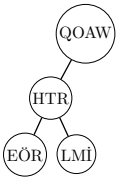
HTR eklendi



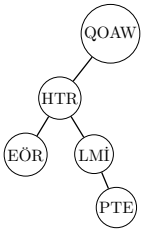
LMİ eklendi



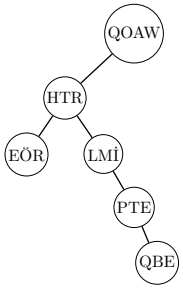
EÖR eklendi



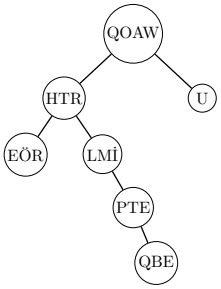
PTE eklendi



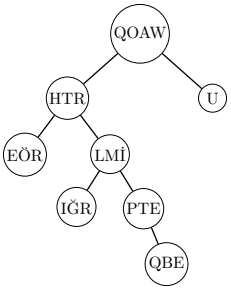
QBE eklendi



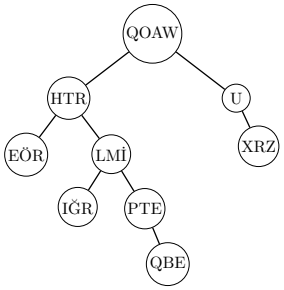
U eklendi



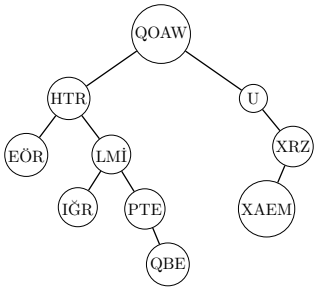
IĞR eklendi



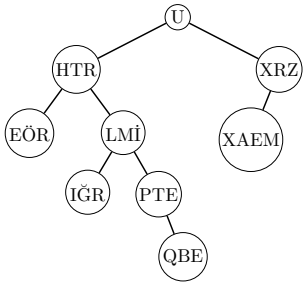
XRZ eklendi



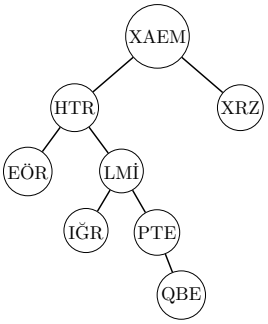
XAEM eklendi



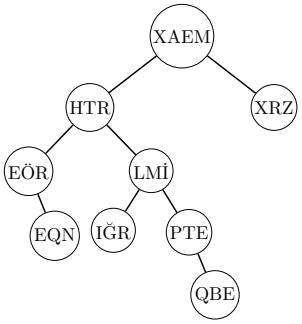
1. kök silme (**QOAW**)
QOAW iki çocuklu: sağ alt ağacın en küçüğü U yerine geçer.



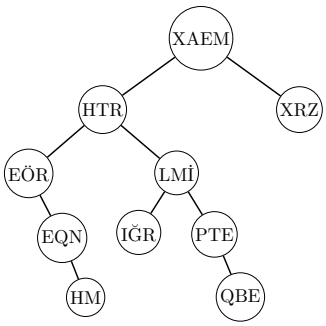
2. kök silme (**U**)
U iki çocuklu: sağ alt ağacın en küçüğü XAEM yerine geçer.



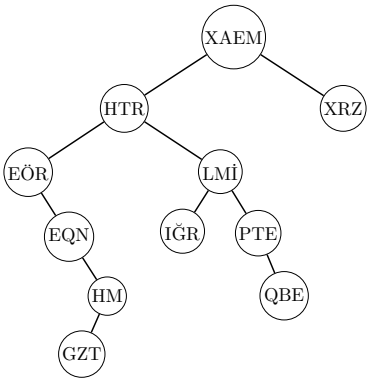
EQN eklendi



HM eklendi



GZT eklendi



3.4 Soru 3 – İfade ağacı ve yığın

Soru 3 – İfade ağacı ve yığın

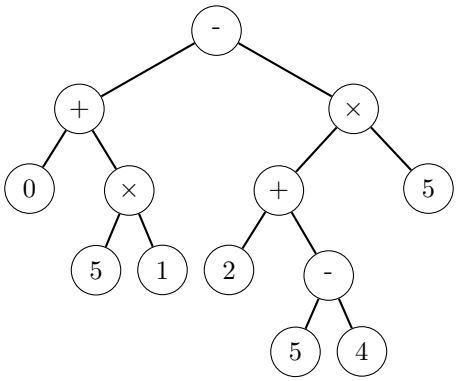
Öğrenci numarasının son 7 rakamı kutulara yazılarak bir sonrakök (postfix) ifadesi elde edilir:

`_ _ _ x + _ _ _ - + _ x -`

Örnek (no: 1030512545): `0 5 1 x + 2 5 4 - + 5 x -`

a) Sonrakök ifadesinin **ağacını** çiziniz. b) İfadeyi hesaplayan **yığın** yapısının her adımını gösteriniz. c) **Öncekök (prefix)** ifadesini yazınız.

a) İfade ağacı (sonek: sol–sağ–kök; sayılar yapraklarda):



b) Yığınla hesaplama (sayı gelince push; işlem gelince iki pop, ilk çıkan SAĞ işlenen):

Adım	İşlem	Yığın (alt → tepe)
1	0 ekle (push)	0
2	5 ekle (push)	0, 5
3	1 ekle (push)	0, 5, 1
4	1 ve 5 çıkar (pop), $5 \times 1 = 5$ ekle	0, 5
5	5 ve 0 çıkar (pop), $0 + 5 = 5$ ekle	5
6	2 ekle (push)	5, 2
7	5 ekle (push)	5, 2, 5
8	4 ekle (push)	5, 2, 5, 4
9	4 ve 5 çıkar (pop), $5 - 4 = 1$ ekle	5, 2, 1
10	1 ve 2 çıkar (pop), $2 + 1 = 3$ ekle	5, 3
11	5 ekle (push)	5, 3, 5
12	5 ve 3 çıkar (pop), $3 \times 5 = 15$ ekle	5, 15
13	15 ve 5 çıkar (pop), $5 - 15 = -10$ ekle	-10

Sonuç: -10

c) Öncekök (prefix) ifadesi: `- + 0 × 5 1 × + 2 - 5 4 5`

İçkök (infix) karşılığı: `((0 + (5 × 1)) - ((2 + (5 - 4)) × 5))`

3.5 Soru 4 – Farz edin ki (kuyruk + yığın + İAA)

Soru 4 – Farz edin ki

Bir **kuyrukta** Kimya, bir **yığında** Fizik, bir **ikili arama ağacında** Biyoloji dersini alan öğrenciler tutulmaktadır (numara, ad, soyad). İkili arama ağacında indeksleme **öğrenci adına** göredir.
a) **Hem Biyoloji hem Fizik** alan öğrencileri listeleyen fonksiyonun kaba kodu (temel kuyruk, yığın ve ağaç işlemleri ile) b) mantığı c) akış diyagramı d) yürütme zamanı ve zaman karmaşıklığı.
Not1: Ağaç hariç diğer yapılarda öğrenciler sıralı değildir. Not2: Sınıf listeleri değişmeyecektir.

İstenen: hem Biyoloji (İAA, *ada göre* indeksli) hem Fizik (yığın) alanlar. Kimya kuyruğu bu soruda kullanılmaz.

a) Kaba kod

```
HemBiyolojiHemFizik(Stack fizik, Tree biyoloji)
 1  Stack gecici                          // tanim: ihmal
 2  while (fizik.isEmpty() == false)
 3     ogr = fizik.pop()                  // tepedeki ogrenciyi al
 4     gecici.push(ogr)                   // kaybetmemek icin gecici yigina koy
```

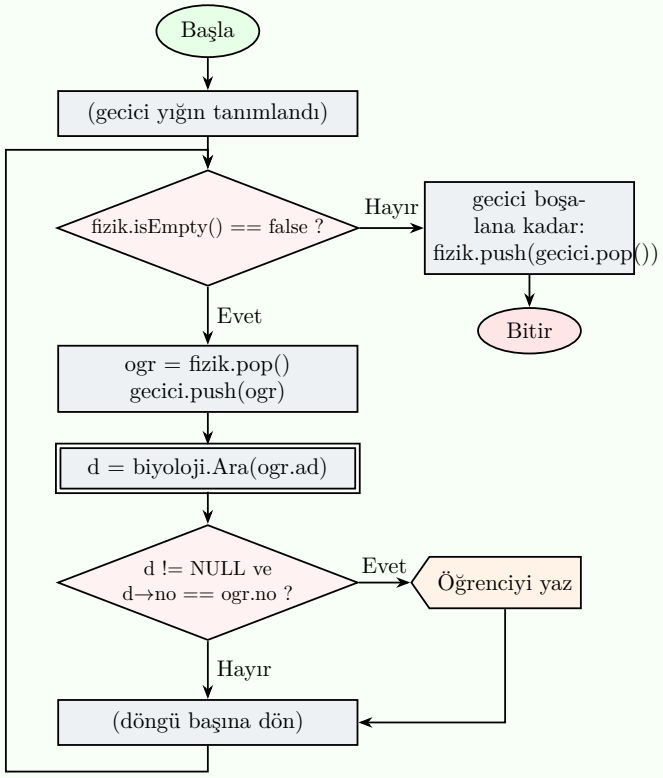
```
 5     d = biyoloji.Ara(ogr.ad)           // IAA'da ada gore arama
 6     if (d != NULL && d->ogr.no == ogr.no)
 7        Yaz(ogr.no, ogr.ad, ogr.soyad)
 8  while (gecici.isEmpty() == false)     // fizik yiginini eski haline getir
 9     fizik.push(gecici.pop())

Node* Ara(Node* kok, string ad)           // IAA arama
     if (kok == NULL) return NULL
     if (ad == kok->ogr.ad) return kok
     if (ad < kok->ogr.ad) return Ara(kok->sol, ad)
     return Ara(kok->sag, ad)
```

b) Mantık

Yığına yalnızca tepeden erişilebildiği için Fizik öğrencileri `pop` ile tek tek alınır ve kaybolmasın diye `gecici` yığına `push` edilir. Her öğrenci Biyoloji ağacında *adına göre* aranır (ağaç ada göre indeksli olduğundan arama sol/sağ seçerek yapılır); ad aynı olan farklı öğrenci olabileceğinden numara da karşılaştırılır. Bulunursa iki dersi de alıyordur, yazdırılır. Sonunda geçici yığındaki öğrenciler tekrar `pop`/`push` edilir: ters çevrilen sıra yeniden ters çevrildiği için Fizik yığını *aynı sırayla* eski haline gelir (Not2).

c) Akış diyagramı



d) Yürütme zamanı ve karmaşıklık

f: Fizik öğrenci sayısı, b: Biyoloji öğrenci sayısı, h: İAA yüksekliği (aramada en fazla $h+1$ düğüm ziyaret edilir, her ziyarette sabit c işlem). En kötü durumda her Fizik öğrencisi Biyoloji'de de

bulunup yazdırılsın:

$$T(f,b) = \underbrace{(f+1)}_{2} + \underbrace{f}_{3} + \underbrace{f}_{4} + \underbrace{f\cdot c(h+1)}_{5} + \underbrace{f}_{6} + \underbrace{f}_{7} + \underbrace{(f+1)}_{8} + \underbrace{2f}_{9} = c\,f\,h + (8+c)f + 2.$$

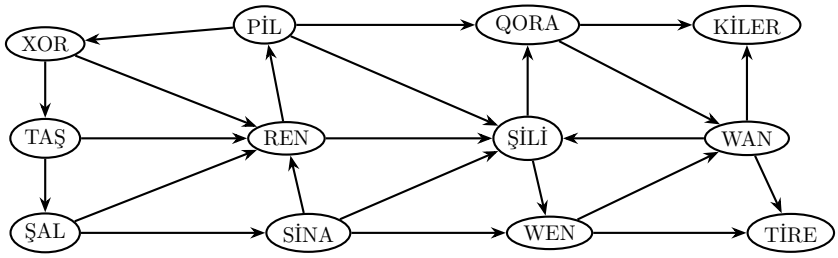
İAA dengesiz olabileceği için en kötü durumda $h = b-1$ (ip gibi dizilmiş ağaç) $\Rightarrow O(f\cdot b)$. Ağaç dengeli ise $h \approx \log b \Rightarrow O(f \log b)$. En iyi durumda (aranan hep kökte) $\Omega(f)$.

4 2024–2025 Güz Vize Çözümleri

4.1 Soru 1 – Genişlik öncelikli arama ve sonrakök (ŞAL'dan, alfabetik sonda öncelikli)

Soru 1 – Graf

Not: Bir düğümün komşuları arasında tercih yapmanız gerektiğinde **alfabetik olarak sonda olanı** önce tercih ediniz (E'nin C'ye önceliği vardır).
a) **ŞAL** düğümünden başlamak şartı ile düğümlerin hangi sırada ziyaret edileceğini **genişlik öncelikli arama** ile listeleyiniz.
b) **ŞAL** düğümünden başlayarak düğümleri **sonrakök (postorder)** mantığına göre sıralayınız.



Komşuluk listesi (komşular alfabetik olarak SONDA olan önce sıralanmış):

KİLER: –

PİL: XOR, ŞİLİ, QORA

QORA: WAN, KİLER

REN: ŞİLİ, PİL

SİNA: WEN, ŞİLİ, REN

ŞAL: SİNA, REN

ŞİLİ: WEN, QORA

TAŞ: ŞAL, REN

TİRE: –

WAN: TİRE, ŞİLİ, KİLER

WEN: WAN, TİRE

XOR: TAŞ, REN

Genişlik öncelikli arama (BFS) — kuyruk kullanılır:

Çıkan	Komşular	Kuyruğa eklenen	Kuyruk
ŞAL	SİNA, REN	SİNA, REN	SİNA, REN
SİNA	WEN, ŞİLİ, REN	WEN, ŞİLİ	REN, WEN, ŞİLİ

REN	ŞİLİ, PİL	PİL	WEN, ŞİLİ, PİL
WEN	WAN, TİRE	WAN, TİRE	ŞİLİ, PİL, WAN, TİRE
ŞİLİ	WEN, QORA	QORA	PİL, WAN, TİRE, QORA
PİL	XOR, ŞİLİ, QORA	XOR	WAN, TİRE, QORA, XOR
WAN	TİRE, ŞİLİ, KİLER	KİLER	TİRE, QORA, XOR, KİLER
TİRE	–	–	QORA, XOR, KİLER
QORA	WAN, KİLER	–	XOR, KİLER
XOR	TAŞ, REN	TAŞ	KİLER, TAŞ
KİLER	–	–	TAŞ
TAŞ	ŞAL, REN	–	boş

BFS ziyaret sırası: ŞAL, SİNA, REN, WEN, ŞİLİ, PİL, WAN, TİRE, QORA, XOR, KİLER, TAŞ

Uzaklıklar (kenar sayısı): ŞAL=0; SİNA=1; REN=1; WEN=2; ŞİLİ=2; PİL=2; WAN=3; TİRE=3; QORA=3; XOR=3; KİLER=4; TAŞ=4

Derinlik öncelikli gezme ve sonrakök (postorder) — bir düğümün bütün (ziyaret edilmemiş) komşuları bitince düğüm listeye yazılır (yığın mantığı):

ŞAL ziyaret edildi, yığın: [ŞAL]

SİNA ziyaret edildi, yığın: [ŞAL, SİNA]

WEN ziyaret edildi, yığın: [ŞAL, SİNA, WEN]

WAN ziyaret edildi, yığın: [ŞAL, SİNA, WEN, WAN]

TİRE ziyaret edildi, yığın: [ŞAL, SİNA, WEN, WAN, TİRE]

TİRE **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, WEN, WAN]

ŞİLİ ziyaret edildi, yığın: [ŞAL, SİNA, WEN, WAN, ŞİLİ]

QORA ziyaret edildi, yığın: [ŞAL, SİNA, WEN, WAN, ŞİLİ, QORA]

KİLER ziyaret edildi, yığın: [ŞAL, SİNA, WEN, WAN, ŞİLİ, QORA, KİLER]

KİLER **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, WEN, WAN, ŞİLİ, QORA]

QORA **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, WEN, WAN, ŞİLİ]

ŞİLİ **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, WEN, WAN]

WAN **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, WEN]

WEN **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA]

REN ziyaret edildi, yığın: [ŞAL, SİNA, REN]

PİL ziyaret edildi, yığın: [ŞAL, SİNA, REN, PİL]

XOR ziyaret edildi, yığın: [ŞAL, SİNA, REN, PİL, XOR]

TAŞ ziyaret edildi, yığın: [ŞAL, SİNA, REN, PİL, XOR, TAŞ]

TAŞ **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, REN, PİL, XOR]

XOR **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, REN, PİL]

PİL **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA, REN]

REN **bitti** ⇒ listeye yaz; yığın: [ŞAL, SİNA]

SİNA **bitti** ⇒ listeye yaz; yığın: [ŞAL]

ŞAL **bitti** ⇒ listeye yaz; yığın: []

DFS (öncekök) ziyaret sırası: ŞAL, SİNA, WEN, WAN, TİRE, ŞİLİ, QORA, KİLER, REN, PİL, XOR, TAŞ

Sonrakök (postorder) sırası: TİRE, KİLER, QORA, ŞİLİ, WAN, WEN, TAŞ, XOR, PİL, REN, SİNA, ŞAL

Dikkat

Postorder (sonrakök) sorusunu hoca düğümden başlayarak DFS mantığıyla istiyor: bir düğümün *bütün* ziyaret edilmemiş komşuları bitince düğüm yazılır. Öğrenci notu: *"Komşuluk listesini (ters) alfabetik olarak oluşturuyorsun, sonra DFS mantığıyla postorder sıralama yapıyorsun."*

4.2 Soru 2a – Hızlı sıralamanın akış diyagramı

Soru 2 – Hızlı sıralama

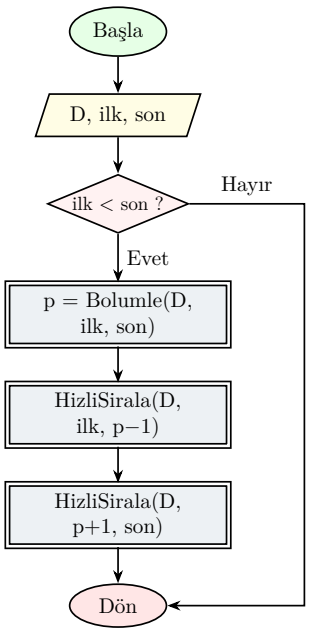
a) Hızlı sıralama algoritmasının **akış diyagramını** çiziniz.
b) Hızlı sıralama algoritmasının **yürütme zamanını ve zaman karmaşıklığını** işlem adımlarını göstererek hesaplayınız.
c) Aşağıdaki diziyi hızlı sıralama ile adım adım sıralayınız. *Pivot seçmeniz gerekirse* ***en baştaki kelimeyi*** *pivot olarak seçiniz.*

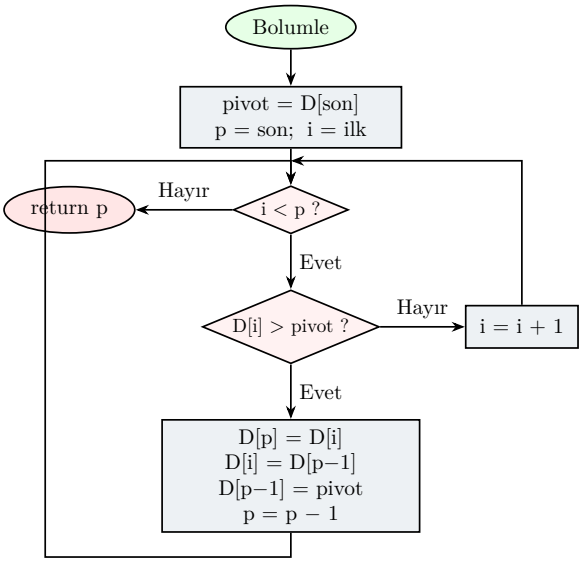
REN, QİM, PER, OTO, TAŞ, QANO, PAS, SARA, ROL, ROT, PARA

Kaba kod (hocanın tek dizi yöntemi; pivot = en sondaki eleman. "En baştaki" kuralında simetrik olarak pivotun sağındakiler soldan sağa taranır):

```
HizliSirala(D, ilk, son)
1  if (ilk < son)
2     p = Bolumle(D, ilk, son)
3     HizliSirala(D, ilk, p - 1)
4     HizliSirala(D, p + 1, son)

Bolumle(D, ilk, son)
5  pivot = D[son];  p = son;  i = ilk
6  while (i < p)
7     if (D[i] > pivot)            // buyukse pivotun sagina gecer
8        D[p] = D[i];  D[i] = D[p-1];  D[p-1] = pivot;  p = p - 1
9     else
10       i = i + 1
11 return p                        // pivot yerini buldu
```





4.3 Soru 2b – Yürütme zamanı ve zaman karmaşıklığı

Çözüm

Bölümleme ($m = son - ilk + 1$ eleman): `while` döngüsünün her turunda ya i artar ya p azalır, bu yüzden döngü tam $m - 1$ tur döner.

Satır	İşlem	En kötü	En iyi
5	3 atama	3	3
6	kontrol	m	m
7	karşılaştırma	$m-1$	$m-1$
8	4 atama	$4(m-1)$	0
10	artırma	0	$m-1$
11	return	1	1

En kötü: $T_B(m) = 3 + m + (m-1) + 4(m-1) + 1 = 6m - 1$; en iyi: $T_B(m) = 3m + 2$. Her iki durumda da bölümleme *doğrusal*dır.

En kötü durum (pivot hep en büyük/en küçük çıkar; hoca: *"En büyük sayıyı pivot seçince en kötü durum olur"*): her bölümlemede tek parça kalır, $n - 1$ elemanlı.

$$T(n) = T(n-1) + T_B(n) + 2 \;\Rightarrow\; T(n) \approx \sum_{m=1}^{n} (6m + 1) = 3n^2 + 4n \;\Rightarrow\; \mathbf{O(n^2)}.$$

En iyi durum (pivot diziyi hep ortadan ikiye böler): $T(n) = 2T(n/2) + 6n$. Bölme $h = \log_2 n$ seviye sürer ($n = 2^h$) ve her seviyede toplam $\approx 6n$ işlem yapılır $\Rightarrow T(n) \approx 6n \log_2 n \Rightarrow$ $(\mathbf{n \log n})$. En iyi ve en kötü durumun dereceleri farklı olduğundan (hocanın kuralına göre) Θ belirtilmez. Alan maliyeti: özyinelemeli çağrılar fonksiyonun kopyalarını oluşturduğu için yüksektir.

4.4 Soru 2c – Hızlı sıralama (pivot: en baştaki kelime)

Pivot kuralı: dizinin (alt dizinin) EN BAŞINDAKİ eleman pivot seçilir; bu kural *her adımda* aynen uygulanır. Pivottan büyükler sağa, küçükler sola geçer. Yer değiştirme hocanın tek dizi yöntemiyle yapılır: eleman pivotun yerine yazılır, pivotun yanındaki eleman boşalan yere yazılır, pivot bir birim kayar.

Başlangıç: REN, QİM, PER, OTO, TAŞ, QANO, PAS, SARA, ROL, ROT, PARA

Alt dizi [0..10], pivot = **REN**

QİM < REN olduğundan pivotun soluna: QİM, **REN**, PER, OTO, TAŞ, QANO, PAS, SARA, ROL, ROT, PARA

PER < REN olduğundan pivotun soluna: QİM, PER, **REN**, OTO, TAŞ, QANO, PAS, SARA, ROL, ROT, PARA

OTO < REN olduğundan pivotun soluna: QİM, PER, OTO, **REN**, TAŞ, QANO, PAS, SARA, ROL, ROT, PARA

QANO < REN olduğundan pivotun soluna: QİM, PER, OTO, QANO, **REN**, TAŞ, PAS, SARA, ROL, ROT, PARA

PAS < REN olduğundan pivotun soluna: QİM, PER, OTO, QANO, PAS, **REN**, TAŞ, SARA, ROL, ROT, PARA

PARA < REN olduğundan pivotun soluna: QİM, PER, OTO, QANO, PAS, PARA, **REN**, SARA, ROL, ROT, TAŞ

Sonuç: [QİM, PER, OTO, QANO, PAS, PARA] **REN** [SARA, ROL, ROT, TAŞ] ⇒ REN

sıralamadaki yerini buldu (indis 6).

Alt dizi [0..5], pivot = **QİM**

PER < QİM olduğundan pivotun soluna: PER, **QİM**, OTO, QANO, PAS, PARA

OTO < QİM olduğundan pivotun soluna: PER, OTO, **QİM**, QANO, PAS, PARA

QANO < QİM olduğundan pivotun soluna: PER, OTO, QANO, **QİM**, PAS, PARA

PAS < QİM olduğundan pivotun soluna: PER, OTO, QANO, PAS, **QİM**, PARA

PARA < QİM olduğundan pivotun soluna: PER, OTO, QANO, PAS, PARA, **QİM**

Sonuç: [PER, OTO, QANO, PAS, PARA] **QİM** [] ⇒ QİM sıralamadaki yerini buldu (indis 5).

Alt dizi [0..4], pivot = **PER**

OTO < PER olduğundan pivotun soluna: OTO, **PER**, QANO, PAS, PARA

PAS < PER olduğundan pivotun soluna: OTO, PAS, **PER**, QANO, PARA

PARA < PER olduğundan pivotun soluna: OTO, PAS, PARA, **PER**, QANO

Sonuç: [OTO, PAS, PARA] **PER** [QANO] ⇒ PER sıralamadaki yerini buldu (indis 3).

Alt dizi [0..2], pivot = **OTO**

Hiçbir eleman yer değiştirmedi (tüm elemanlar zaten doğru tarafta).

Sonuç: [] **OTO** [PAS, PARA] ⇒ OTO sıralamadaki yerini buldu (indis 0).

Alt dizi [1..2], pivot = **PAS**

PARA < PAS olduğundan pivotun soluna: PARA, **PAS**

Sonuç: [PARA] **PAS** [] ⇒ PAS sıralamadaki yerini buldu (indis 2).

▷ [1..1] alt dizisinde tek eleman (PARA) kaldı ⇒ yerini buldu.

▷ [4..4] alt dizisinde tek eleman (QANO) kaldı ⇒ yerini buldu.

Alt dizi [7..10], pivot = **SARA**

ROL < SARA olduğundan pivotun soluna: ROL, **SARA**, ROT, TAŞ

ROT < SARA olduğundan pivotun soluna: ROL, ROT, **SARA**, TAŞ

Sonuç: [ROL, ROT] **SARA** [TAŞ] ⇒ SARA sıralamadaki yerini buldu (indis 9).

Alt dizi [7..8], pivot = **ROL**

Hiçbir eleman yer değiştirmedi (tüm elemanlar zaten doğru tarafta).

Sonuç: [] **ROL** [ROT] ⇒ ROL sıralamadaki yerini buldu (indis 7).

▷ [8..8] alt dizisinde tek eleman (ROT) kaldı ⇒ yerini buldu.

▷ [10..10] alt dizisinde tek eleman (TAŞ) kaldı ⇒ yerini buldu.

Sıralı dizi: OTO, PARA, PAS, PER, QANO, QİM, REN, ROL, ROT, SARA, TAŞ

4.5 Soru 3 – B+ ağacı (iç düğüm anahtar sayısı 3, yaprak maks. 3 eleman)

Soru 3 – B+ ağacı

a) Yukarıdaki kelime dizisini bir B+ ağacına verildiği sırada yerleştiriniz. **Ağacın iç düğüm anahtar sayısı ve yapraklarda tutulan maksimum eleman sayısı 3'tür.**
b) a şıkkında oluşan ağaçtan sırasıyla **PER** ve **OTO** silindiğinde değişimleri gösteriniz; son halini çiziniz.

"İç düğüm anahtar sayısı 3" $\Rightarrow M-1=3$, $M=4$; "yapraklarda tutulan maksimum eleman 3" $\Rightarrow L-1=3$, $L=4$. Parametreler: iç düğüm çocuk sayısı $M=4 \Rightarrow$ iç düğümde en fazla $M-1=3$, en az $\lceil M/2 \rceil - 1 = 1$ anahtar; $L=4 \Rightarrow$ yaprakta en fazla $L-1=3$, en az $\lceil L/2 \rceil - 1 = 1$ veri.

Başlangıç ağacı



REN ekle: REN yaprağa eklendi (yaprakta boş yer vardı).



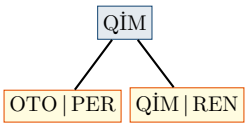
QİM ekle: QİM yaprağa eklendi (yaprakta boş yer vardı).



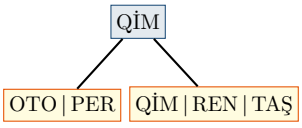
PER ekle: PER yaprağa eklendi (yaprakta boş yer vardı).



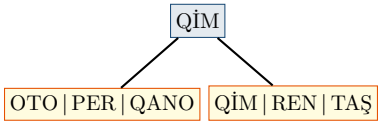
OTO ekle: OTO eklenince yaprakta 4 veri oldu (maks. 3) $\Rightarrow$ yaprak bölünür: sola $\lfloor 4/2 \rfloor$=2 en küçük (OTO, PER), sağa 2 (QİM, REN); sağdakinin en küçüğü QİM yukarı anahtar olarak KOPYALANIR. Kök bölündüğü için yeni kök oluştu: [QİM].



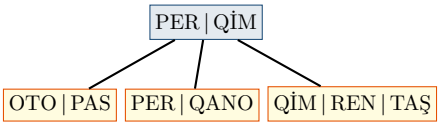
TAŞ ekle: TAŞ yaprağa eklendi (yaprakta boş yer vardı).



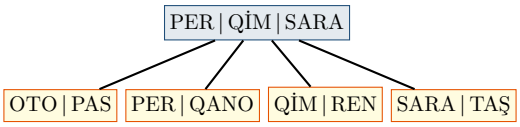
QANO ekle: QANO yaprağa eklendi (yaprakta boş yer vardı).



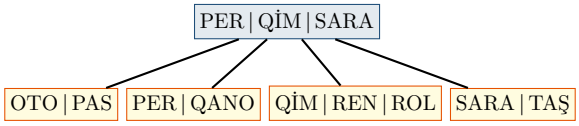
PAS ekle: PAS eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor = 2$ en küçük (OTO, PAS), sağa 2 (PER, QANO); sağdakinin en küçüğü PER yukarı anahtar olarak KOPYALANIR.



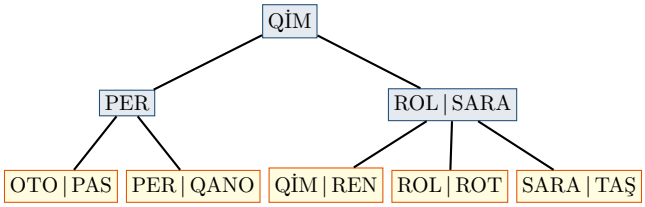
SARA ekle: SARA eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor = 2$ en küçük (QİM, REN), sağa 2 (SARA, TAŞ); sağdakinin en küçüğü SARA yukarı anahtar olarak KOPYALANIR.



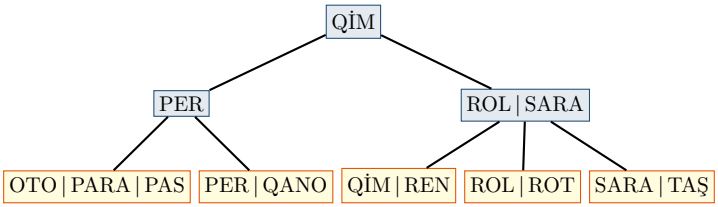
ROL ekle: ROL yaprağa eklendi (yaprakta boş yer vardı).



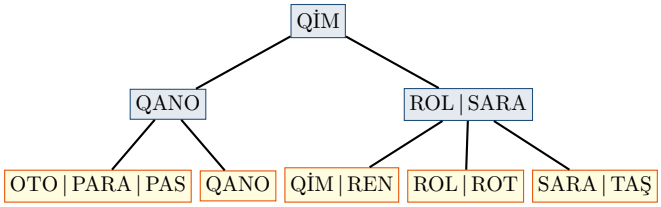
ROT ekle: ROT eklenince yaprakta 4 veri oldu (maks. 3) ⇒ yaprak bölünür: sola $\lfloor 4/2 \rfloor = 2$ en küçük (QİM, REN), sağa 2 (ROL, ROT); sağdakinin en küçüğü ROL yukarı anahtar olarak KOPYALANIR. İç düğümde 4 anahtar oldu (maks. 3) ⇒ iç düğüm bölünür: sola $\lceil 4/2 \rceil - 1 = 1$ anahtar (PER), QİM yukarı TAŞINIR, sağa ROL, SARA. Kök bölündüğü için yeni kök oluştu: [QİM].



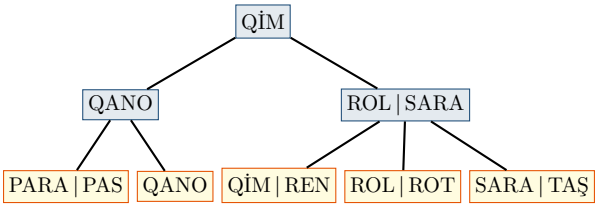
PARA ekle: PARA yaprağa eklendi (yaprakta boş yer vardı).



PER sil: PER yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



OTO sil: OTO yapraktan silindi. Minimum kural sağlanıyor; gerekirse üst anahtarlar sağdakinin en küçüğü ile güncellenir.



4.6 Soru 4 – Farz edin ki (AVL + yığın + bağlantılı liste): sadece bir ders alanlar

Soru 4 – Farz edin ki

Bir **AVL ağacında** Lojik, bir **yığında** Olasılık, bir **bağlantılı listede** İngilizce dersini alan öğrenciler tutulmaktadır (numara, ad, soyad). Bazı öğrenciler birden fazla ders alabilmektedir. a) **Sadece bir ders alan** öğrencileri bulan fonksiyonun kaba kodu (temel AVL, yığın ve bağlantılı liste işlemleri ile) b) akış diyagramı ve mantığı c) yürütme zamanı ve zaman karmaşıklığı. *Not1: Yığın ve bağlantılı listede öğrenciler sıralı değildir. Not2: Sınıf listeleri değişmeyecektir.*

AVL ağacı (Lojik) soruda belirtilmediği için *öğrenci numarasına göre* indeksli kabul edilmiştir (cevapta bu varsayım yazılmalıdır).

a) Kaba kod

```
bool YigindaVarMi(Stack y, int no)        // yigin bozulmadan aranir
   Stack gec;  var = false
   while (y.isEmpty() == false)
      o = y.pop();  gec.push(o)
      if (o.no == no) var = true
   while (gec.isEmpty() == false)  y.push(gec.pop())
   return var

bool ListedeVarMi(List l, int no)
   d = l.head
   while (d != NULL)
      if (d->ogr.no == no) return true
      d = d->next
   return false

SadeceBirDers(AVL lojik, Stack olasilik, List ingilizce)
 // 1) Sadece Lojik alanlar: AVL ickok (inorder) gezilir
 IckokGez(lojik.kok):  her o icin
     if (!YigindaVarMi(olasilik, o.no) && !ListedeVarMi(ingilizce, o.no))  Yaz(o)
 // 2) Sadece Olasilik alanlar
 Stack gec
 while (olasilik.isEmpty() == false)
     o = olasilik.pop();  gec.push(o)
     if (lojik.Ara(o.no) == NULL && !ListedeVarMi(ingilizce, o.no))  Yaz(o)
 while (gec.isEmpty() == false)  olasilik.push(gec.pop())
 // 3) Sadece Ingilizce alanlar
 d = ingilizce.head
 while (d != NULL)
     if (lojik.Ara(d->ogr.no) == NULL && !YigindaVarMi(olasilik, d->ogr.no))  Yaz(d->ogr)
     d = d->next
```

b) Mantık

Bir öğrencinin "sadece bir ders" alması, bulunduğu yapı dışındaki *iki yapıda da bulunmaması* demektir. Bu yüzden her yapı sırayla gezilir ve öğrenci diğer ikisinde aranır. AVL'de arama numaraya göre dengeli ağaçta yapıldığından hızlıdır (log); yığın ve liste sırasız olduğu için doğrusal aranır. Yığında arama yapılırken elemanlar geçici yığına aktarılıp geri konur; böylece hiçbir sınıf listesi değişmez.

c) Yürütme zamanı ve karmaşıklık

a: Lojik, y: Olasılık, l: İngilizce öğrenci sayısı. `YigindaVarMi` $\approx 1+(y+1)+3y+y+(y+1)+2y = 8y+3$, `ListedeVarMi` en kötü $\approx 3l+2$, AVL araması $\approx c\log a$.

$$T \approx a\,(8y+3l)+y\,(c\log a+3l)+l\,(c\log a+8y)+O(a+y+l) = 8ay+3al+11yl+c(y+l)\log a+\ldots$$

En yüksek dereceli terimler çarpımlardır: $O(ay+al+yl)$; üç sınıf da yaklaşık n kişiyse $\mathbf{O(n^2)}$.

5 2024–2025 Mazeret Sınavı Çözümleri

5.1 Soru 1 – fonk2 ve fonk3

Soru 1 – Yürütme zamanı ve zaman karmaşıklığı

Aşağıdaki fonksiyonların i) yürütme zamanlarını ve ii) zaman karmaşıklıklarını hesaplayınız.

```
void fonk2(int n) {
  j = 0;
  while (j < n) {
    for (int i = 0; i < n; ++i) {
      System.out.println("i = " + i);
      for (int k = 0; k < i; ++k)
        System.out.println("k = " + k);
    }
    j = j + 1;
  }
}
```

```
void fonk3(int n) {
  for (int i = 0; i < n * n; ++i)
    for (int k = 0; k < i; ++k)
      System.out.println("k = " + k);
  for (int j = n; j > 0; j--)
    System.out.println("j = " + j);
}
```

fonk2

`while` n kez döner. Her turda dış `for` ($i = 0..n-1$): atama 1, kontrol $n+1$, artırma $n \Rightarrow 2n+2$. İç `for` ($k = 0..i-1$) i için $1+(i+1)+i = 2i+2$ işlem; $\sum_{i=0}^{n-1}(2i+2) = n^2+n$. "k" yazdırma $\sum i = \frac{n(n-1)}{2}$.

Satır	Tekrar
`j = 0`	1
`while (j < n)`	$n+1$
dış `for`	$n(2n+2)$
`println(i)`	$n \cdot n$
iç `for`	$n(n^2+n)$
`println(k)`	$n \cdot \frac{n(n-1)}{2}$
`j = j + 1`	n

$$T(n) = 1+(n+1)+2n^2+2n+n^2+n^3+n^2+\tfrac{n^3-n^2}{2}+n = \mathbf{\tfrac{3}{2}n^3+\tfrac{7}{2}n^2+4n+2}$$

Koşul (if) olmadığı için en iyi = en kötü: $O(n^3)$, $\Omega(n^3)$, $\Theta(n^3)$.

fonk3

Dış `for` $i = 0..n^2-1$: $1+(n^2+1)+n^2 = 2n^2+2$. İç `for`: $\sum_{i=0}^{n^2-1}(2i+2) = n^2(n^2-1)+2n^2 = n^4+n^2$. "k" yazdırma: $\sum_{i=0}^{n^2-1} i = \frac{n^2(n^2-1)}{2}$. İkinci döngü ($j = n..1$): $1 + (n+1) + n = 2n+2$; yazdırma n.

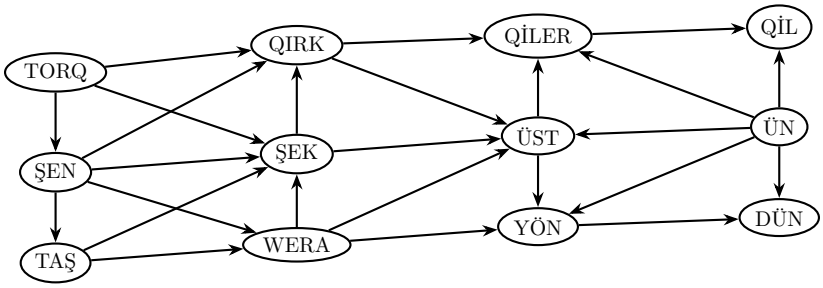
$$T(n) = (2n^2+2) + (n^4+n^2) + \tfrac{n^4-n^2}{2} + (2n+2) + n = \mathbf{\tfrac{3}{2}n^4 + \tfrac{5}{2}n^2 + 3n + 4}$$

$\Rightarrow O(n^4)$, $\Omega(n^4)$, $\Theta(n^4)$. (Hocanın uyarısı: *"for görünce direkt doğrusalı veya kareseli yapıştırmamak lazım"* – dış döngü n^2 kez dönüyor.)

5.2 Soru 2 – TORQ'dan BFS ve sonrakök

Soru 2 – Graf

Şekildeki graf kullanılacaktır. Komşular arasında seçimde **alfabetik olarak sonda olana** öncelik veriniz.
a) **TORQ** düğümünden başlayarak genişlik öncelikli arama ile ziyaret sırasını listeleyiniz. b) **TORQ** düğümünden başlayarak düğümleri sonrakök (postorder) mantığına göre sıralayınız.



Komşuluk listesi (komşular alfabetik olarak SONDA olan önce sıralanmış):

DÜN: –

QIRK: ÜST, QİLER

QİL: –

QİLER: QİL

ŞEK: ÜST, QIRK

ŞEN: WERA, TAŞ, ŞEK, QIRK

TAŞ: WERA, ŞEK

TORQ: ŞEN, ŞEK, QIRK

ÜN: YÖN, ÜST, QİLER, QİL, DÜN

ÜST: YÖN, QİLER

WERA: YÖN, ÜST, ŞEK

YÖN: DÜN

Genişlik öncelikli arama (BFS) — kuyruk kullanılır:

Çıkan	Komşular	Kuyruğa eklenen	Kuyruk
TORQ	ŞEN, ŞEK, QIRK	ŞEN, ŞEK, QIRK	ŞEN, ŞEK, QIRK
ŞEN	WERA, TAŞ, ŞEK, QIRK	WERA, TAŞ	ŞEK, QIRK, WERA, TAŞ

ŞEK	ÜST, QIRK	ÜST	QIRK, WERA, TAŞ, ÜST
QIRK	ÜST, QİLER	QİLER	WERA, TAŞ, ÜST, QİLER
WERA	YÖN, ÜST, ŞEK	YÖN	TAŞ, ÜST, QİLER, YÖN
TAŞ	WERA, ŞEK	–	ÜST, QİLER, YÖN
ÜST	YÖN, QİLER	–	QİLER, YÖN
QİLER	QİL	QİL	YÖN, QİL
YÖN	DÜN	DÜN	QİL, DÜN
QİL	–	–	DÜN
DÜN	–	–	boş

BFS ziyaret sırası: TORQ, ŞEN, ŞEK, QIRK, WERA, TAŞ, ÜST, QİLER, YÖN, QİL, DÜN

Uzaklıklar (kenar sayısı): TORQ=0; ŞEN=1; ŞEK=1; QIRK=1; WERA=2; TAŞ=2; ÜST=2; QİLER=2; YÖN=3; QİL=3; DÜN=4

Derinlik öncelikli gezme ve sonrakök (postorder) — bir düğümün bütün (ziyaret edilmemiş) komşuları bitince düğüm listeye yazılır (yığın mantığı):

TORQ ziyaret edildi, yığın: [TORQ]

ŞEN ziyaret edildi, yığın: [TORQ, ŞEN]

WERA ziyaret edildi, yığın: [TORQ, ŞEN, WERA]

YÖN ziyaret edildi, yığın: [TORQ, ŞEN, WERA, YÖN]

DÜN ziyaret edildi, yığın: [TORQ, ŞEN, WERA, YÖN, DÜN]

DÜN **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA, YÖN]

YÖN **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA]

ÜST ziyaret edildi, yığın: [TORQ, ŞEN, WERA, ÜST]

QİLER ziyaret edildi, yığın: [TORQ, ŞEN, WERA, ÜST, QİLER]

QİL ziyaret edildi, yığın: [TORQ, ŞEN, WERA, ÜST, QİLER, QİL]

QİL **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA, ÜST, QİLER]

QİLER **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA, ÜST]

ÜST **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA]

ŞEK ziyaret edildi, yığın: [TORQ, ŞEN, WERA, ŞEK]

QIRK ziyaret edildi, yığın: [TORQ, ŞEN, WERA, ŞEK, QIRK]

QIRK **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA, ŞEK]

ŞEK **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN, WERA]

WERA **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN]

TAŞ ziyaret edildi, yığın: [TORQ, ŞEN, TAŞ]

TAŞ **bitti** ⇒ listeye yaz; yığın: [TORQ, ŞEN]

ŞEN **bitti** ⇒ listeye yaz; yığın: [TORQ]

TORQ **bitti** ⇒ listeye yaz; yığın: []

DFS (öncekök) ziyaret sırası: TORQ, ŞEN, WERA, YÖN, DÜN, ÜST, QİLER, QİL, ŞEK, QIRK, TAŞ

Sonrakök (postorder) sırası: DÜN, YÖN, QİL, QİLER, ÜST, QIRK, ŞEK, WERA, TAŞ, ŞEN, TORQ

Not: ÜN düğüm(ler)ine TORQ'den yol yoktur, bu yüzden listeye girmez.

6 2025–2026 Güz Vize Çözümleri (16 Kasım 2025)

Kelimeler

Sınavın orijinal kelime listesi klasörde yok. Çözüm, sınavdaki durumla (ikinci kök silmede en sağdaki XOR'da *sağ-sağ* problemi ve sola dönüş) birebir aynı davranan örnek kelimelerle yapılmıştır: **ARI, DAL, KAR, PİR, XOR, GÜL, YOL, ZİL, NAR, JET**.

6.1 Soru 1 – Hızlı sıralama

Soru 1 – Hızlı sıralama (Quick Sort)

a) Hızlı sıralamanın **akış diyagramını** çiziniz. b) Hızlı sıralamanın **karmaşıklık analizini** göstererek hesaplayınız (yürütme zamanı + zaman karmaşıklığı). c) Verilen **kelimeleri** hızlı sıralama ile sıralayınız.

a) ve b): 2024–25 Vize Soru 2a ve 2b'deki akış diyagramı ve yürütme zamanı analizi aynen geçerlidir ($O(n^2)$ en kötü, $\Omega(n \log n)$ en iyi).

c) Sıralama (pivot: en sondaki kelime):

Pivot kuralı: dizinin (alt dizinin) EN SONUNDAKİ eleman pivot seçilir; bu kural *her adımda* aynen uygulanır. Pivottan büyükler sağa, küçükler sola geçer. Yer değiştirme hocanın tek dizi yöntemiyle yapılır: eleman pivotun yerine yazılır, pivotun yanındaki eleman boşalan yere yazılır, pivot bir birim kayar.

Başlangıç: ARI, DAL, KAR, PİR, XOR, GÜL, YOL, ZİL, NAR, JET

Alt dizi [0..9], pivot = **JET**

KAR > JET olduğundan pivotun sağına: ARI, DAL, NAR, PİR, XOR, GÜL, YOL, ZİL, **JET**, KAR

NAR > JET olduğundan pivotun sağına: ARI, DAL, ZİL, PİR, XOR, GÜL, YOL, **JET**, NAR, KAR

ZİL > JET olduğundan pivotun sağına: ARI, DAL, YOL, PİR, XOR, GÜL, **JET**, ZİL, NAR, KAR

YOL > JET olduğundan pivotun sağına: ARI, DAL, GÜL, PİR, XOR, **JET**, YOL, ZİL, NAR, KAR

PİR > JET olduğundan pivotun sağına: ARI, DAL, GÜL, XOR, **JET**, PİR, YOL, ZİL, NAR, KAR

XOR > JET olduğundan pivotun sağına: ARI, DAL, GÜL, **JET**, XOR, PİR, YOL, ZİL, NAR, KAR

Sonuç: [ARI, DAL, GÜL] **JET** [XOR, PİR, YOL, ZİL, NAR, KAR] ⇒ JET sıralamadaki yerini buldu (indis 3).

Alt dizi [0..2], pivot = **GÜL**

Hiçbir eleman yer değiştirmedi (tüm elemanlar zaten doğru tarafta).

Sonuç: [ARI, DAL] **GÜL** [] ⇒ GÜL sıralamadaki yerini buldu (indis 2).

Alt dizi [0..1], pivot = **DAL**

Hiçbir eleman yer değiştirmedi (tüm elemanlar zaten doğru tarafta).

Sonuç: [ARI] **DAL** [] ⇒ DAL sıralamadaki yerini buldu (indis 1).

▷ [0..0] alt dizisinde tek eleman (ARI) kaldı ⇒ yerini buldu.

Alt dizi [4..9], pivot = **KAR**

XOR > KAR olduğundan pivotun sağına: NAR, PİR, YOL, ZİL, **KAR**, XOR

NAR > KAR olduğundan pivotun sağına: ZİL, PİR, YOL, **KAR**, NAR, XOR

ZİL > KAR olduğundan pivotun sağına: YOL, PİR, **KAR**, ZİL, NAR, XOR

YOL > KAR olduğundan pivotun sağına: PİR, **KAR**, YOL, ZİL, NAR, XOR

PİR > KAR olduğundan pivotun sağına: **KAR**, PİR, YOL, ZİL, NAR, XOR

Sonuç: [] **KAR** [PİR, YOL, ZİL, NAR, XOR] ⇒ KAR sıralamadaki yerini buldu (indis 4).

Alt dizi [5..9], pivot = **XOR**

YOL > XOR olduğundan pivotun sağına: PİR, NAR, ZİL, **XOR**, YOL

ZİL > XOR olduğundan pivotun sağına: PİR, NAR, **XOR**, ZİL, YOL

Sonuç: [PİR, NAR] **XOR** [ZİL, YOL] ⇒ XOR sıralamadaki yerini buldu (indis 7).

Alt dizi [5..6], pivot = **NAR**

PİR > NAR olduğundan pivotun sağına: **NAR**, PİR

Sonuç: [] **NAR** [PİR] ⇒ NAR sıralamadaki yerini buldu (indis 5).

▷ [6..6] alt dizisinde tek eleman (PİR) kaldı ⇒ yerini buldu.

Alt dizi [8..9], pivot = **YOL**

ZİL > YOL olduğundan pivotun sağına: **YOL**, ZİL

Sonuç: [] **YOL** [ZİL] ⇒ YOL sıralamadaki yerini buldu (indis 8).

▷ [9..9] alt dizisinde tek eleman (ZİL) kaldı ⇒ yerini buldu.

Sıralı dizi: ARI, DAL, GÜL, JET, KAR, NAR, PİR, XOR, YOL, ZİL

6.2 Soru 2 – İkili arama ağacında silme

Soru 2 – İkili ağaç (silme)

İkili (arama) ağaçta **veri silme fonksiyonunun** akış diyagramını, kodunu (kaba kod) ve karmaşıklık analizini yapınız. Öğrenci aktarımına göre bir alt şıkta ağaç üzerinde *"sol taraf → sağ taraf → düğüm"* sırasıyla (sonrakök) gezme istenmiştir.

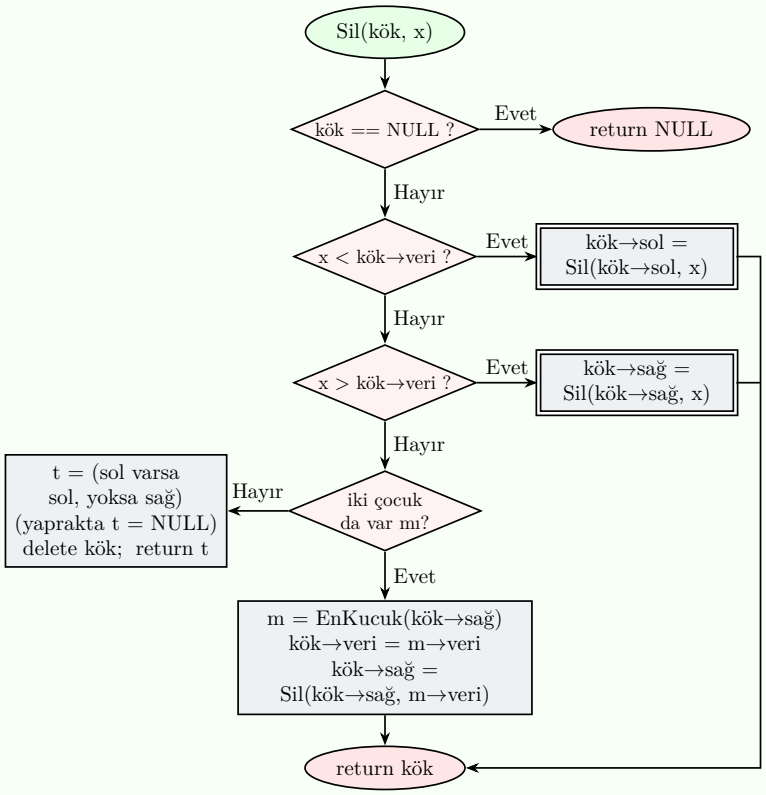
Kaba kod

```
Node* Sil(Node* kok, x)
 1  if (kok == NULL) return NULL                        // bulunamadi
 2  if (x < kok->veri)       kok->sol = Sil(kok->sol, x)
 3  else if (x > kok->veri)  kok->sag = Sil(kok->sag, x)
 4  else                                                 // silinecek dugum bulundu
 5     if (kok->sol == NULL && kok->sag == NULL)        // 1) yaprak
 6        delete kok;  return NULL
 7     else if (kok->sol == NULL || kok->sag == NULL)  // 2) tek cocuk
 8        t = (kok->sol != NULL) ? kok->sol : kok->sag
 9        delete kok;  return t                         // cocuk dedeye baglanir
10     else                                             // 3) iki cocuk
11        m = EnKucuk(kok->sag)                         // sag alt agacin en kucugu
```

```
12        kok->veri = m->veri
13        kok->sag = Sil(kok->sag, m->veri)
14  return kok

Node* EnKucuk(Node* d)
15  while (d->sol != NULL)  d = d->sol                  // en sola git
16  return d
```

Akış diyagramı



Yürütme zamanı ve karmaşıklık

Silme, kökten silinecek düğüme giden yol boyunca her seviyede sabit sayıda işlem yapar: satır 1 (kontrol), 2–3 (en fazla 2 karşılaştırma + 1 atama/çağrı) $\Rightarrow$ seviye başına ≈ 4 işlem. Silinecek düğüm d derinliğindeyse arama $4(d+1)$; iki çocuklu durumda `EnKucuk` sağ alt ağaçta en fazla $h-d$ adım (satır 15: kontrol + atama ≈ 2 işlem) ve ardından ikinci silme aynı yolu izler.

$$T_{\text{en kötü}}(h) \approx 4(h+1) + 2h + 6 \approx 6h + 10 \quad (h : \text{ağaç yüksekliği}).$$

İkili arama ağacı dengeli değilse veriler sıralı gelince "ip gibi" dizilir, $h = n-1$ olur: $O(n)$. Dengeli ağaçta (ör. AVL) $h \approx \log n \Rightarrow O(\log n)$. En iyi durum: silinecek kök ve tek çocuklu/yaprak $\Rightarrow \Omega(1)$.

Sonrakök (sol–sağ–kök) gezme kodu

```
Sonrakok(Node* d)
   if (d == NULL) return
   Sonrakok(d->sol)      // once sol taraf
   Sonrakok(d->sag)      // sonra sag taraf
   Yaz(d->veri)          // en son dugumun kendisi
```

Her düğüm bir kez ziyaret edilir: $T(n) \approx 4n + 1 \Rightarrow \Theta(n)$.

6.3 Soru 3 – Farz edin ki (liste + kuyruk + AVL): birden fazla ders alanlar

Soru 3 – Farz edin ki

Üç farklı ders: biri **liste** (bağlantılı liste), biri **kuyruk**, biri **AVL ağacı** yapısında tutuluyor. **Birden fazla ders alan** öğrencileri (her bölüm/ders için "çok ders alanlar") bulan ve yazdıran fonksiyonun kaba kodu, mantığı, akış diyagramı, yürütme zamanı/karmaşıklığı. Veri yapıları korunacaktır.

"Çok ders" ne demek?

Sınav sonrası derste hoca sorulduğunda: *"Çok ile en çok aynı şey" / "o zaman 1'den fazla alanları bulun"* dedi (öğrenci aktarımı, 17.11.2025). Yani **en az iki ders alanlar**.

Liste: A dersi, kuyruk: B dersi, AVL (numaraya göre): C dersi.

a) Kaba kod

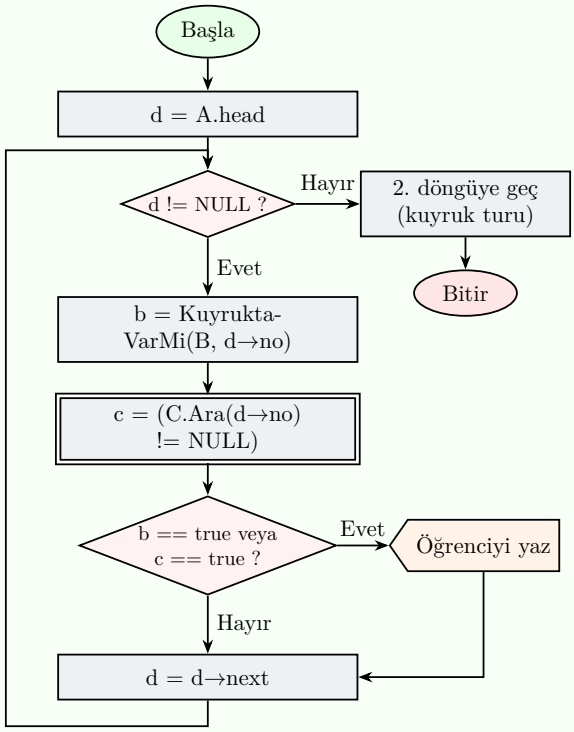
```
bool KuyruktaVarMi(Queue q, int no)          // kuyruk bir tur dondurulur
   var = false;  boyut = q.size()
   for (i = 0; i < boyut; i++)
      o = q.dequeue()
      if (o.no == no) var = true
      q.enqueue(o)                           // ayni sirayla geri ekle
   return var

BirdenFazlaDersAlanlar(List A, Queue B, AVL C)
 // 1) Listedeki her ogrenci: B veya C'de de varsa en az 2 ders alir
 d = A.head
 while (d != NULL)
    b = KuyruktaVarMi(B, d->ogr.no)
    c = (C.Ara(d->ogr.no) != NULL)
    if (b == true || c == true)  Yaz(d->ogr)
    d = d->next
 // 2) Listede OLMAYAN ama kuyrukta ve AVL'de olanlar (tekrar yazmamak icin)
 boyut = B.size()
 for (i = 0; i < boyut; i++)
    o = B.dequeue()
    if (!ListedeVarMi(A, o.no) && C.Ara(o.no) != NULL)  Yaz(o)
    B.enqueue(o)
```

b) Mantık

En az iki ders alan bir öğrenci ya listede (A) vardır ve B ya da C'den en az birinde de bulunur, ya da A'da yoktur ama hem B'de hem C'de vardır. Birinci döngü ilk grubu, ikinci döngü ikinci grubu bulur; böylece her öğrenci *bir kez* yazdırılır. Kuyrukta arama `dequeue`/`enqueue` ile kuyruğun boyutu kadar döndürülerek yapılır; tur sonunda kuyruk aynı sıraya döner. Listeden ve AVL'den hiçbir şey silinmez ⇒ yapılar korunur. (Hoca derste `size()`, `clone()`, `Search()` gibi temel fonksiyonları kullandı; kendi yazdığınız yardımcı fonksiyonun kodunu/mantığını mutlaka verin.)

c) Akış diyagramı (1. döngü)



d) Yürütme zamanı ve karmaşıklık

l: liste, q: kuyruk, a: AVL öğrenci sayısı. `KuyruktaVarMi` $\approx 2 + (2q + 2) + 4q = 6q + 4$; AVL arama $c \log a$; `ListedeVarMi` $\leq 3l + 2$.

$T \approx l\,(6q + c\log a + 8) + q\,(3l + c\log a + 6) + 4 = 9lq + c(l+q)\log a + 8l + 6q + 4 \;\Rightarrow\; \mathbf{O(l \cdot q)}\ (\approx O(n^2))$.

6.4 Soru 4 – AVL ağacı

Soru 4 – AVL ağacı

a) Verilen kelimeleri sırasıyla **AVL ağacına** yerleştiriniz. b) **Kökü iki kez** siliniz (bazı öğrencilerde ikinci silmede denge bozuldu; *"en sağda XOR vardı, sola döndürmen lazımdı"*). c) **Sonrakök** gezme yapınız.

Ekleme adımları (her eklemeden sonra eklenen düğümden köke doğru denge kontrolü):

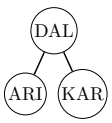
ARI eklendi
denge var



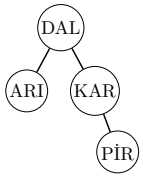
DAL eklendi
denge var



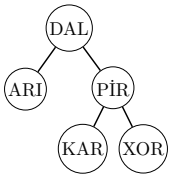
KAR eklendi
ARI düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ ARI'den sola tek dönüş.



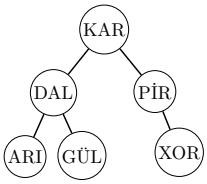
PİR eklendi
denge var



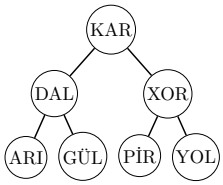
XOR eklendi
KAR düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ KAR'den sola tek dönüş.



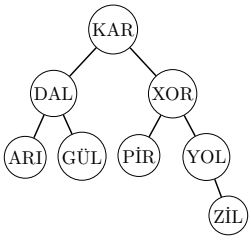
GÜL eklendi
DAL düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce PİR'den sağa, sonra DAL'den sola.



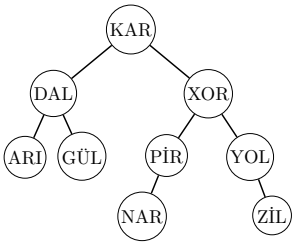
YOL eklendi
PİR düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ PİR'den sola tek dönüş.



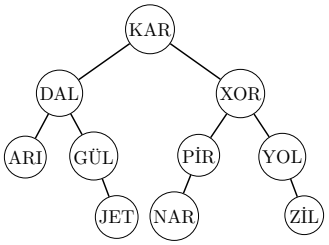
ZİL eklendi
denge var



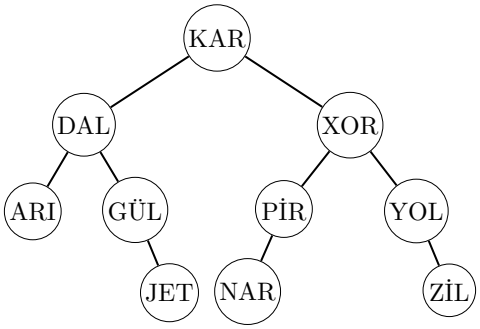
NAR eklendi
denge var



JET eklendi
denge var

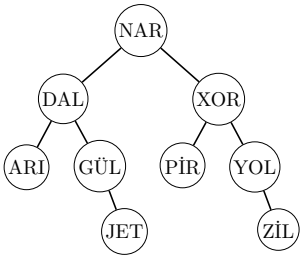


Ağacın son hali (kontrol: içkök gezme sıralı verir: ARI, DAL, GÜL, JET, KAR, NAR, PİR, XOR, YOL, ZİL):

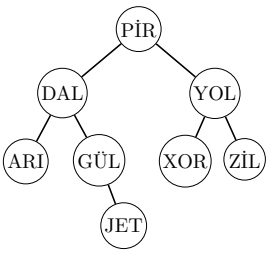


Silme adımları (iki çocuklu düğüm silinirken *sağ alt ağacın en küçüğü* yerine geçer; ardından denge kontrolü):

1. kök silme: **KAR**
KAR iki çocuklu: sağ alt ağacın en küçüğü NAR köke/yerine geçer. Ardından NAR sağ alt ağaçtaki eski yerinden silinir: NAR yaprak, doğrudan silinir. Denge bozulmadı.



2. kök silme: **NAR**
NAR iki çocuklu: sağ alt ağacın en küçüğü PİR köke/yerine geçer. Ardından PİR sağ alt ağaçtaki eski yerinden silinir: PİR yaprak, doğrudan silinir. XOR düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ XOR'den sola tek dönüş.



Sonrakök (postorder: sol–sağ–kök) gezme (son ağaç üzerinde): **ARI, JET, GÜL, DAL, XOR, ZİL, YOL, PİR**

7 Yılı bilinmeyen vize çözümleri

Aktarılan vize

1) **Hızlı sıralama**: akış diyagramı, karmaşıklık analizi, kelime dizisini sıralama.
2) **AVL**: ağaç oluşturma, **iki kez kök silme**, **öncekök** gezme ile çıkarma.
3) **Bağlantılı listeye ekleme** fonksiyonunun akış diyagramı ve karmaşıklığı (öğrenci: *"hoca yaz okulunda bağlantılı liste ekleme fonksiyonuyla ilgili soru sormuş... Bizden istedi"* – yani fonksiyonu öğrenci yazar).
4) **Farz edin ki**: yığın, kuyruk ve AVL ile.

7.1 Soru 1 – Hızlı sıralama (örnek kelimeler, pivot en sondaki)

Akış diyagramı ve karmaşıklık analizi için 2024–25 Vize Soru 2'ye bakınız. Benzer kelimelerle (hocanın derste tahtaya yazdığı "vaka, vana, vane..." tarzı) sıralama:

Pivot kuralı: dizinin (alt dizinin) EN SONUNDAKİ eleman pivot seçilir; bu kural *her adımda* aynen uygulanır. Pivottan büyükler sağa, küçükler sola geçer. Yer değiştirme hocanın tek dizi yöntemiyle yapılır: eleman pivotun yerine yazılır, pivotun yanındaki eleman boşalan yere yazılır, pivot bir birim kayar.

Başlangıç: VAKA, VANA, VANE, VONA, VAN, VON, VONG, VAND, VANI, VONB

Alt dizi [0..9], pivot = **VONB**

VONG > VONB olduğundan pivotun sağına: VAKA, VANA, VANE, VONA, VAN, VON, VANI, VAND, **VONB**, VONG

Sonuç: [VAKA, VANA, VANE, VONA, VAN, VON, VANI, VAND] **VONB** [VONG] ⇒ VONB sıralamadaki yerini buldu (indis 8).

Alt dizi [0..7], pivot = **VAND**

VANE > VAND olduğundan pivotun sağına: VAKA, VANA, VANI, VONA, VAN, VON, **VAND**, VANE

VANI > VAND olduğundan pivotun sağına: VAKA, VANA, VON, VONA, VAN, **VAND**, VANI, VANE

VON > VAND olduğundan pivotun sağına: VAKA, VANA, VAN, VONA, **VAND**, VON, VANI, VANE

VONA > VAND olduğundan pivotun sağına: VAKA, VANA, VAN, **VAND**, VONA, VON, VANI, VANE

Sonuç: [VAKA, VANA, VAN] **VAND** [VONA, VON, VANI, VANE] ⇒ VAND sıralamadaki yerini buldu (indis 3).

Alt dizi [0..2], pivot = **VAN**

VANA > VAN olduğundan pivotun sağına: VAKA, **VAN**, VANA

Sonuç: [VAKA] **VAN** [VANA] ⇒ VAN sıralamadaki yerini buldu (indis 1).

▷ [0..0] alt dizisinde tek eleman (VAKA) kaldı ⇒ yerini buldu.

▷ [2..2] alt dizisinde tek eleman (VANA) kaldı ⇒ yerini buldu.

Alt dizi [4..7], pivot = **VANE**

VONA > VANE olduğundan pivotun sağına: VANI, VON, **VANE**, VONA

VANI > VANE olduğundan pivotun sağına: VON, **VANE**, VANI, VONA

VON > VANE olduğundan pivotun sağına: **VANE**, VON, VANI, VONA

Sonuç: [] **VANE** [VON, VANI, VONA] ⇒ VANE sıralamadaki yerini buldu (indis 4).

Alt dizi [5..7], pivot = **VONA**

Hiçbir eleman yer değiştirmedi (tüm elemanlar zaten doğru tarafta).

Sonuç: [VON, VANI] **VONA** [] ⇒ VONA sıralamadaki yerini buldu (indis 7).

Alt dizi [5..6], pivot = **VANI**

VON > VANI olduğundan pivotun sağına: **VANI**, VON

Sonuç: [] **VANI** [VON] ⇒ VANI sıralamadaki yerini buldu (indis 5).

▷ [6..6] alt dizisinde tek eleman (VON) kaldı ⇒ yerini buldu.

▷ [9..9] alt dizisinde tek eleman (VONG) kaldı ⇒ yerini buldu.

Sıralı dizi: VAKA, VAN, VANA, VAND, VANE, VANI, VON, VONA, VONB, VONG

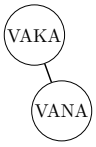
7.2 Soru 2 – AVL oluşturma, iki kök silme, öncekök

Ekleme adımları (her eklemeden sonra eklenen düğümden köke doğru denge kontrolü):

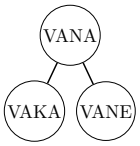
VAKA eklendi
denge var



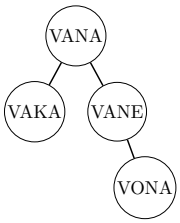
VANA eklendi
denge var



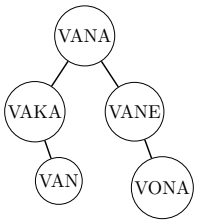
VANE eklendi
VAKA düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ VAKA'den sola tek dönüş.



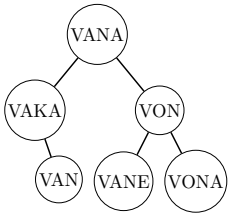
VONA eklendi
denge var



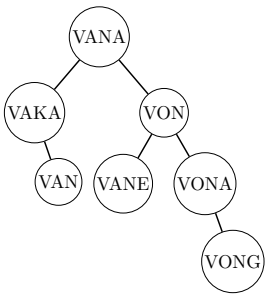
VAN eklendi
denge var



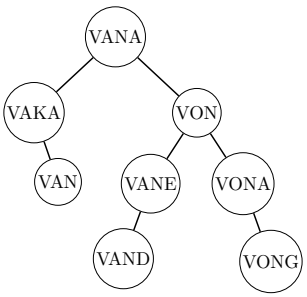
VON eklendi
VANE düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce VONA'den sağa, sonra VANE'den sola.



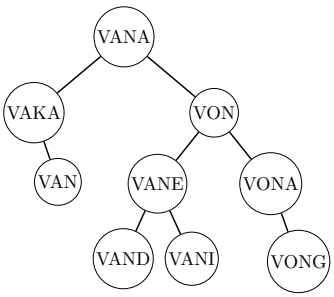
VONG eklendi
denge var



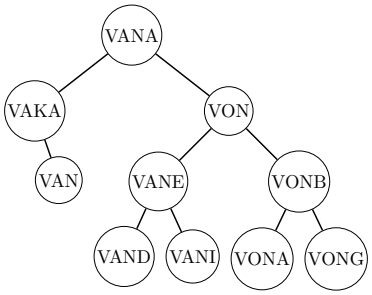
VAND eklendi
denge var



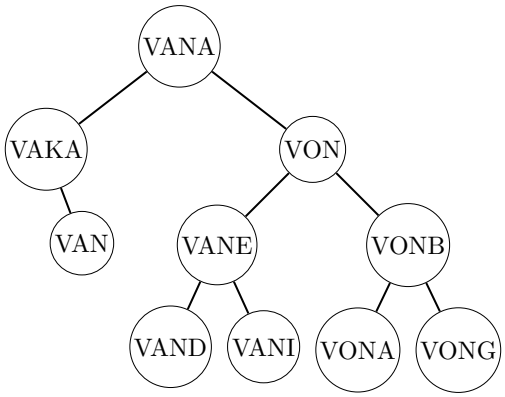
VANI eklendi
denge var



VONB eklendi
VONA düğümünde denge bozuldu:
SAĞ-SOL problemi ⇒ çift dönüş: önce
VONG'den sağa, sonra VONA'den sola.

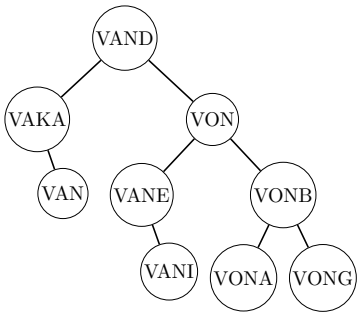


Ağacın son hali (kontrol: içkök gezme sıralı verir: VAKA, VAN, VANA, VAND, VANE, VANI, VON, VONA, VONB, VONG):

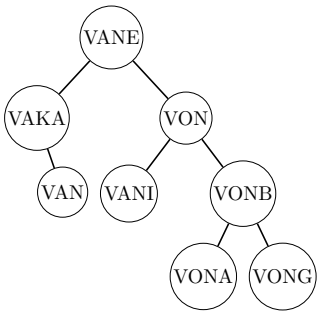


Silme adımları (iki çocuklu düğüm silinirken *sağ alt ağacın en küçüğü* yerine geçer; ardından denge kontrolü):

1. kök silme: **VANA**
VANA iki çocuklu: sağ alt ağacın en küçüğü VAND köke/yerine geçer. Ardından VAND sağ alt ağaçtaki eski yerinden silinir: VAND yaprak, doğrudan silinir. Denge bozulmadı.



2. kök silme: **VAND**
VAND iki çocuklu: sağ alt ağacın en küçüğü VANE köke/yerine geçer. Ardından VANE sağ alt ağaçtaki eski yerinden silinir: VANE tek çocuklu, çocuğu VANI yukarı bağlanır. Denge bozulmadı.



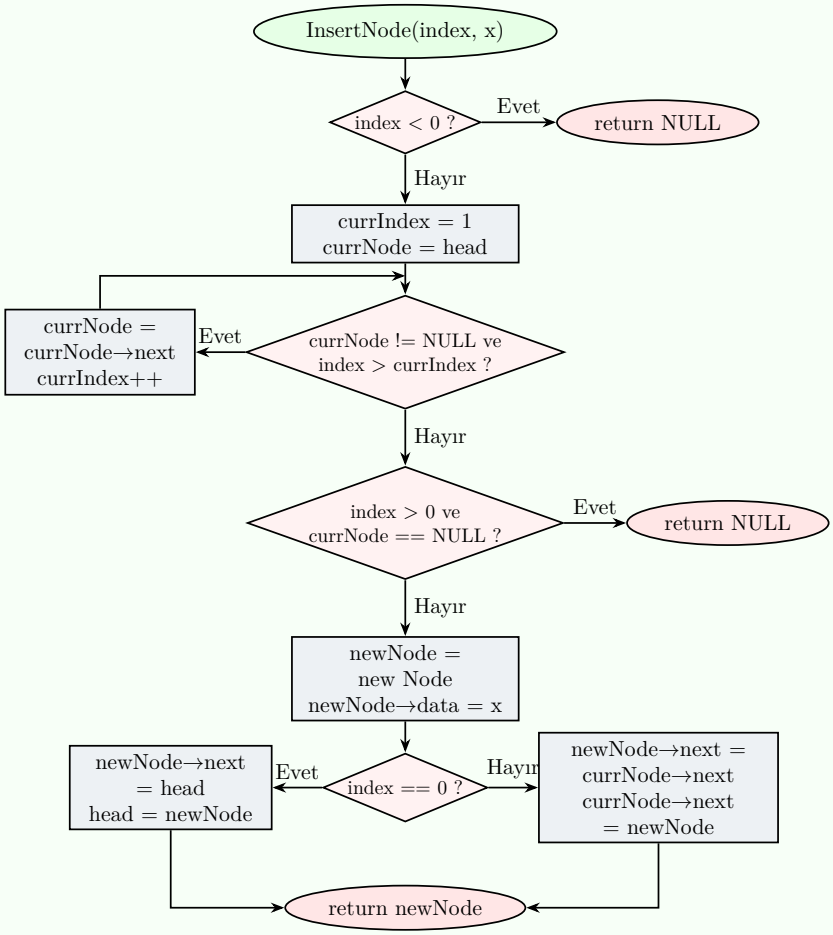
Öncekök (preorder: kök–sol–sağ) gezme (son ağaç üzerinde): **VANE, VAKA, VAN, VON, VANI, VONB, VONA, VONG**

7.3 Soru 3 – Bağlantılı listeye ekleme (InsertNode)

Kaba kod (ders slaytındaki fonksiyon)

```
Node* List::InsertNode(int index, double x) {
1   if (index < 0) return NULL;
2   int currIndex = 1;
3   Node* currNode = head;
4   while (currNode && index > currIndex) {   // index'inci dugumu bul
5       currNode = currNode->next;
6       currIndex++;
    }
7   if (index > 0 && currNode == NULL) return NULL;
8   Node* newNode = new Node;
9   newNode->data = x;
10  if (index == 0) {                         // basa ekle
11      newNode->next = head;
12      head = newNode;
    } else {                                  // currNode'dan sonra ekle
13      newNode->next = currNode->next;
14      currNode->next = newNode;
    }
15  return newNode;
}
```

Akış diyagramı



Yürütme zamanı ve karmaşıklık

n: listedeki düğüm sayısı. En kötü durum: sona ekleme ($index = n$), `while` $n-1$ kez döner.

Satır	En kötü	En iyi (başa ekleme)
1	1	1
2, 3	2	2
4	n	1
5, 6	$2(n-1)$	0
7	1	1
8, 9	2	2
10	1	1
11–14	2	2
15	1	1

$T_{\text{en kötü}}(n) = 1+2+n+2(n-1)+1+2+1+2+1 = 3n+8 \Rightarrow O(n)$; $T_{\text{en iyi}} = 11 \Rightarrow \Omega(1)$ (sabit). Dereceler farklı $\Rightarrow \Theta$ yok. (Dizi ile gerçekleştirmede başa ekleme tüm elemanları kaydırdığı için $O(n)$ idi; bağlantılı listede başa ekleme sabittir.)

7.4 Soru 4 – Farz edin ki (yığın + kuyruk + AVL): üç dersi de alanlar

Hocanın derste tahtada çözdüğü yaklaşım (fotoğraf `HocaFarzEdinKiCozum.jpeg`): yığından `pop`, AVL'de `find` ($\log n$), kuyrukta `dequeue`–`enqueue` turu.

Çözüm

```
UcDersiAlanlar(Stack Y, Queue K, AVL T)       // T numaraya gore indeksli
 1  Stack Z
 2  for (i = 1; i <= Y.size(); i++)          // ya da: while(!Y.isEmpty())
 3     A = Y.pop();  Z.push(A)
 4     if (T.Find(A.no) != NULL)              // log n
 5        for (j = 1; j <= K.size(); j++)
 6           B = K.dequeue()
 7           if (A.no == B.no)  Yaz(A)
 8           K.enqueue(B)
 9  while (Z.isEmpty() == false)  Y.push(Z.pop())
```

y, k, t sınıf mevcutları: en kötü $T \approx y(c\log t) + y\cdot(4k+2) + 4y + \cdots \Rightarrow O(y\cdot k + y\log t)$; hepsi n ise $O(n^2)$. AVL'de olmayan öğrenci için kuyruk hiç taranmadığından en iyi durum $\Omega(y\log t)$.

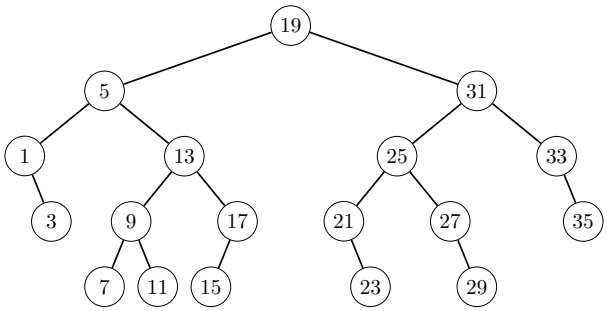
8 Hocanın Örnek Sınav Soruları Çözümleri

8.1 Örnek 1 – Şekil 1 AVL: 8 ekle, 2 ekle, 9 sil

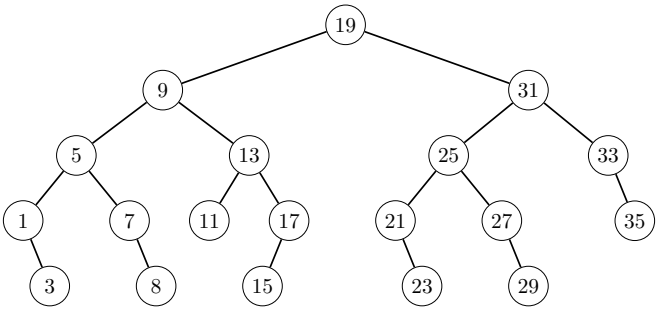
Örnek 1 – AVL (Şekil 1)

Şekil 1'deki AVL ağacı: kök 19; 5 (1 (–, 3), 13 (9 (7, 11), 17 (15, –))); 31 (25 (21 (–, 23), 27 (–, 29)), 33 (–, 35)).
a) 8 eklenirse ağacın son hali? b) a'da oluşan ağaca 2 eklenirse? c) b'de oluşan ağaçtan 9 silinirse?

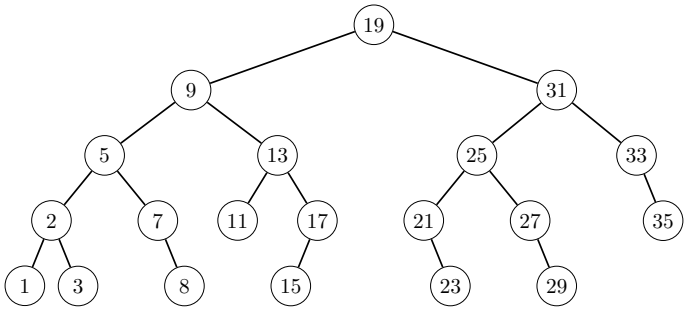
Şekil 1



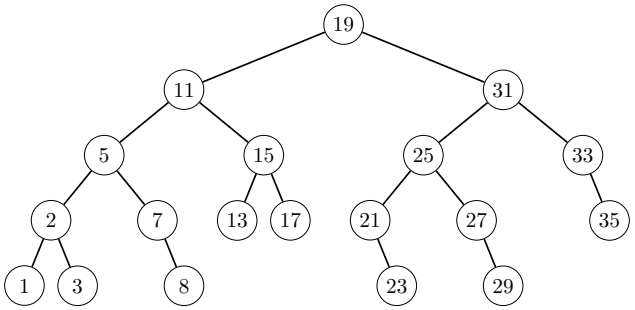
a) 8 ekle: 5 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 13'den sağa, sonra 5'den sola.



b) 2 ekle: 1 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 3'den sağa, sonra 1'den sola.



c) 9 sil: 9 iki çocuklu: sağ alt ağacın en küçüğü 11 köke/yerine geçer. Ardından 11 sağ alt ağaçtaki eski yerinden silinir: 11 yaprak, doğrudan silinir. 13 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 17'den sağa, sonra 13'den sola.

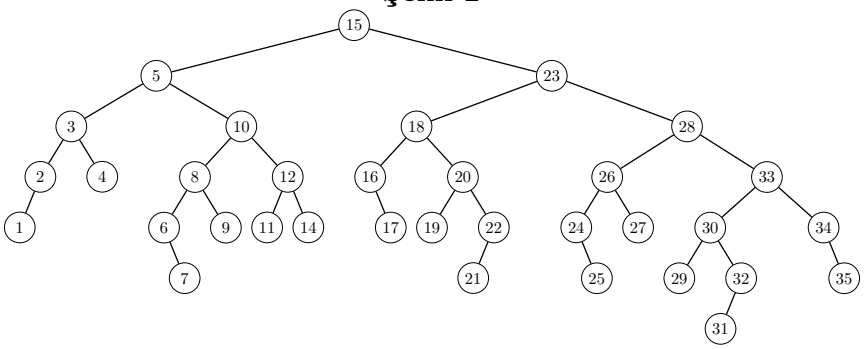


8.2 Örnek 2 – Şekil 2 AVL: 4 sil; kök sil

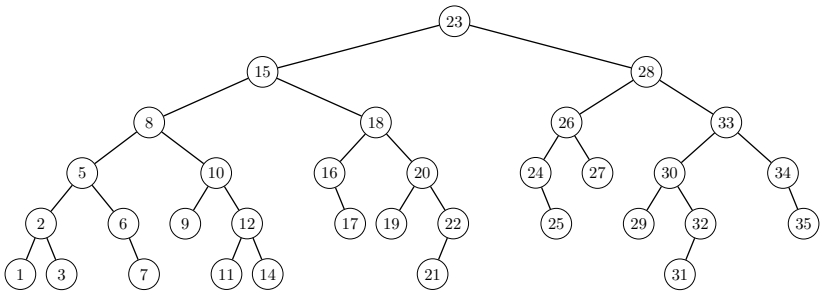
Örnek 2 – AVL (Şekil 2)

Şekil 2: kök 15; 5 (3 (2 (1), 4), 10 (8 (6 (–, 7), 9), 12 (11, 14))); 23 (18 (16 (–, 17), 20 (19, 22 (21, –))), 28 (26 (24 (–, 25), 27), 33 (30 (29, 32 (31, –)), 34 (–, 35)))).
a) 4 silinirse ağacın son hali? b) Kök silinirse ağacın son hali?

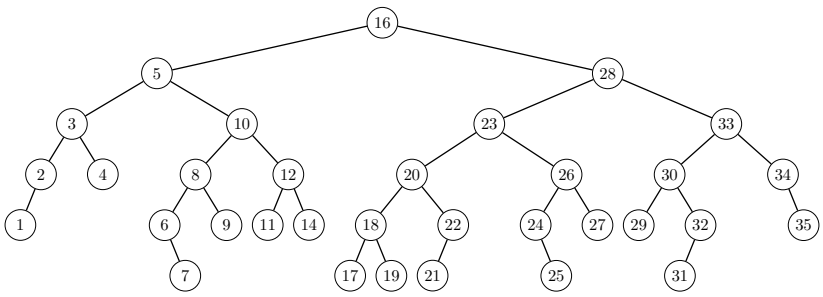
Şekil 2



a) 4 sil (Şekil 2'deki orijinal ağaçtan): 4 yaprak, doğrudan silinir. 3 düğümünde denge bozuldu: SOL-SOL problemi ⇒ 3'den sağa tek dönüş. 5 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 10'dan sağa, sonra 5'den sola. 15 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 15'den sola tek dönüş.



b) 15 sil (Şekil 2'deki orijinal ağaçtan): 15 iki çocuklu: sağ alt ağacın en küçüğü 16 köke/yerine geçer. Ardından 16 sağ alt ağaçtaki eski yerinden silinir: 16 tek çocuklu, çocuğu 17 yukarı bağlanır. 18 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 18'den sola tek dönüş. 23 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 23'den sola tek dönüş.



8.3 Örnek 3 – Graf: DFS ve BFS (A'dan, alfabetik önde öncelikli)

Örnek 3 – Graf

Kenarlar: A→B, A→D, B→F, C→A, C→B, C→D, C→F, D→E, E→C, E→F, E→G, E→I, G→F, H→B, H→F, I→G, I→H.
a) A'dan başlayarak DFS sıralama sonucu (en az 4 adım). b) A'dan başlayarak BFS sıralama sonucu. (Komşular arasında **alfabetik önde olan** tercih edilir.)

Komşuluk listesi (komşular alfabetik olarak ÖNDE olan önce sıralanmış):

A: B, D

B: F

C: A, B, D, F

D: E

E: C, F, G, I

F: –

G: F

H: B, F

I: G, H

Genişlik öncelikli arama (BFS) — kuyruk kullanılır:

Çıkan	Komşular	Kuyruğa eklenen	Kuyruk
A	B, D	B, D	B, D
B	F	F	D, F
D	E	E	F, E
F	–	–	E
E	C, F, G, I	C, G, I	C, G, I
C	A, B, D, F	–	G, I
G	F	–	I
I	G, H	H	H
H	B, F	–	boş

BFS ziyaret sırası: A, B, D, F, E, C, G, I, H

Uzaklıklar (kenar sayısı): A=0; B=1; D=1; F=2; E=2; C=3; G=3; I=3; H=4

Derinlik öncelikli gezme ve sonrakök (postorder) — bir düğümün bütün (ziyaret edilmemiş) komşuları bitince düğüm listeye yazılır (yığın mantığı):

A ziyaret edildi, yığın: [A]

B ziyaret edildi, yığın: [A, B]

F ziyaret edildi, yığın: [A, B, F]

F **bitti** ⇒ listeye yaz; yığın: [A, B]

B **bitti** ⇒ listeye yaz; yığın: [A]

D ziyaret edildi, yığın: [A, D]

E ziyaret edildi, yığın: [A, D, E]

C ziyaret edildi, yığın: [A, D, E, C]

C **bitti** ⇒ listeye yaz; yığın: [A, D, E]

G ziyaret edildi, yığın: [A, D, E, G]

G **bitti** ⇒ listeye yaz; yığın: [A, D, E]

I ziyaret edildi, yığın: [A, D, E, I]

H ziyaret edildi, yığın: [A, D, E, I, H]

H **bitti** ⇒ listeye yaz; yığın: [A, D, E, I]

I **bitti** ⇒ listeye yaz; yığın: [A, D, E]

E **bitti** ⇒ listeye yaz; yığın: [A, D]

D **bitti** ⇒ listeye yaz; yığın: [A]

A **bitti** ⇒ listeye yaz; yığın: []

DFS (öncekök) ziyaret sırası: A, B, F, D, E, C, G, I, H

Sonrakök (postorder) sırası (ek bilgi): F, B, C, G, H, I, E, D, A

8.4 Örnek 4 – AVL (sayılar)

Örnek 4 – AVL (sayılar)

Boş bir AVL ağacına sırası ile 10, 30, 25, 20, 70, 50, 40, 35, 30, 60, 45 yerleştirilirse: a) son hali (adım adım), b) kök silinirse, c) ardından 25 silinirse ağacın son hali?

Eşit değer (ikinci 30) için kural: *eşitse sağa* (hoca: *"bir kural belirleyip ona sadık kalacağız"*). **Ekleme adımları** (her eklemeden sonra eklenen düğümden köke doğru denge kontrolü):

10 eklendi
denge var

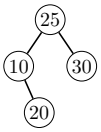
30 eklendi
denge var



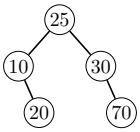
25 eklendi
10 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 30'den sağa, sonra 10'den sola.



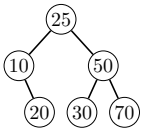
20 eklendi
denge var



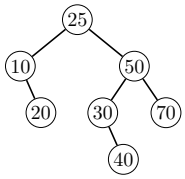
70 eklendi
denge var



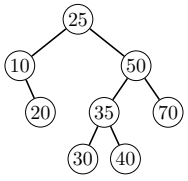
50 eklendi
30 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 70'den sağa, sonra 30'den sola.



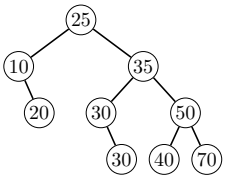
40 eklendi
denge var



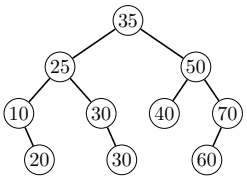
35 eklendi
30 düğümünde denge bozuldu: SAĞ-SOL problemi ⇒ çift dönüş: önce 40'den sağa, sonra 30'den sola.



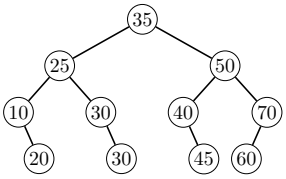
30 eklendi
50 düğümünde denge bozuldu: SOL-SOL problemi ⇒ 50'den sağa tek dönüş.



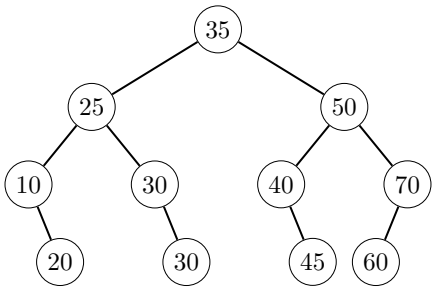
60 eklendi
25 düğümünde denge bozuldu: SAĞ-SAĞ problemi ⇒ 25'den sola tek dönüş.



45 eklendi
denge var

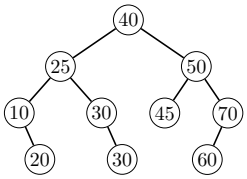


Ağacın son hali (kontrol: içkök gezme sıralı verir: 10, 20, 25, 30, 30, 35, 40, 45, 50, 60, 70):

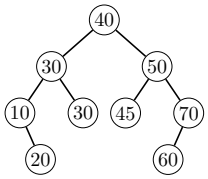


Silme adımları (iki çocuklu düğüm silinirken *sağ alt ağacın en küçüğü* yerine geçer; ardından denge kontrolü):

1. kök silme: **35**
35 iki çocuklu: sağ alt ağacın en küçüğü 40 köke/yerine geçer. Ardından 40 sağ alt ağaçtaki eski yerinden silinir: 40 tek çocuklu, çocuğu 45 yukarı bağlanır. Denge bozulmadı.



25 silindi
25 iki çocuklu: sağ alt ağacın en küçüğü 30 köke/yerine geçer. Ardından 30 sağ alt ağaçtaki eski yerinden silinir: 30 tek çocuklu, çocuğu 30 yukarı bağlanır. Denge bozulmadı.

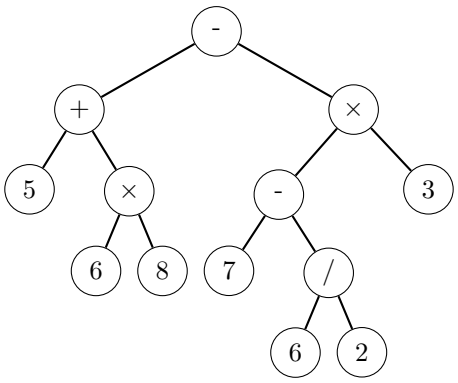


8.5 Örnek 5 – Postfix

Örnek 5 – İfade ağacı ve yığın

`5 6 8 * + 7 6 2 / - 3 * -` sonrakök ifadesinin a) ağacını çiziniz, b) yığınla hesaplamanın her adımını gösteriniz, c) içkök (infix) ifadesini yazınız.

a) İfade ağacı:



b) Yığınla hesaplama:

Adım	İşlem	Yığın (alt → tepe)
1	5 ekle (push)	5
2	6 ekle (push)	5, 6
3	8 ekle (push)	5, 6, 8
4	8 ve 6 çıkar (pop), $6 \times 8 = 48$ ekle	5, 48
5	48 ve 5 çıkar (pop), $5 + 48 = 53$ ekle	53
6	7 ekle (push)	53, 7
7	6 ekle (push)	53, 7, 6
8	2 ekle (push)	53, 7, 6, 2

9	2 ve 6 çıkar (pop), $6/2 = 3$ ekle	53, 7, 3
10	3 ve 7 çıkar (pop), $7 - 3 = 4$ ekle	53, 4
11	3 ekle (push)	53, 4, 3
12	3 ve 4 çıkar (pop), $4 \times 3 = 12$ ekle	53, 12
13	12 ve 53 çıkar (pop), $53 - 12 = 41$ ekle	41

Sonuç: 41

c) İçkök (infix): `((5 + (6 × 8)) - ((7 - (6 / 2)) × 3))`

8.6 Örnek 6 – Farz edin ki (liste + kuyruk): her iki dersi alanlar

Örnek 6 – Farz edin ki

Bir **bağlantılı listede** Veri Yapıları, bir **kuyrukta** Programlama dersini alan öğrenciler tutulmaktadır. a) **Her iki dersi de** alan öğrencileri bulan fonksiyonun kaba kodu, b) mantığı, c) akış diyagramı. (Sırasız; listeler değişmeyecek.)

Çözüm

```
IkiDersiAlanlar(List VY, Queue P)
 1  boyut = P.size()
 2  for (i = 0; i < boyut; i++)
 3     o = P.dequeue()                  // kuyrugun onundeki ogrenci
 4     d = VY.head
 5     while (d != NULL)
 6        if (d->ogr.no == o.no) { Yaz(o); break }
 7        d = d->next
 8     P.enqueue(o)                     // ayni ogrenciyi arkaya geri koy
```

Mantık: Kuyruktan sadece önden alınıp arkaya eklenebildiği için kuyruk boyutu kadar `dequeue`–`enqueue` yapılır; tur bitince kuyruk eski sırasına döner. Her öğrenci listede doğrusal aranır. **Karmaşıklık:** p kuyruk, v liste boyutu: $T \approx p(3v + 5) + 3 \Rightarrow O(p \cdot v)$.